

2026 暑期学校 · 深度学习

深度学习基础

Deep Learning Basics

魏鸿鑫

南方科技大学 统计与数据科学系

教材: Bishop & Bishop, Deep Learning (2024)

讲者

魏鸿鑫

Hongxin Wei

南方科技大学 统计与数据科学系 助理教授 · 博导 | 深圳河套学院 客座教授

hongxin001.github.io

研究方向

可靠机器学习 · 自动化评估 · 搜索智能体

模型不只要预测得准，还要说得出「我有多确定」

形式：置信度校准 · conformal prediction

如：模块 02 的 Softmax 置信度、模块 05 的 Label Smoothing 校准

学术服务与产出

Area Chair

NeurIPS · ICML · ICLR

期刊审稿

JASA · JMLR · TPAMI

近年发表

合计56篇论文 · 20余篇三大会一作/通讯
Cranking机器学习领域南科大PI排名第一

开源

TorchCP · ChineseSafe · SearcherKit

课题组招人

长期招研究实习生 / RA

同时接收硕博申请

本科生也可申请实习

往届成员去向

腾讯青云计划 · NUS PhD · NTU
PhD · SUNY Buffalo
PhD · MBZUAI PhD · CUHK(SZ)
MPhil

课题组介绍

团队成员:

博士生 6人

硕士生 3人

研究助理 4 人

(来自南科大、电子科大、大连理工、
澳门大学、哈工深等高校)

计算资源

- 计算集群:
 - 60 余张企业级 GPU (含英伟达 A100、Pro 6000、L40、4090)
 - 长期租赁60余张 A100显卡
 - 企业合作 (H200、B200等显卡)
- 自有机房: 自建 UPS 与精密空调系统, 确保集群 7*24 小时稳定运行。



课题组风格

1. 理论与前沿并进

我们研究适用于LLM时代的统计理论, 也深入设计搜索智能体等前沿内容, 为学生提供论文与开源项目等多重机会;

2. 鼓励LLM学生在企业开展研究

所有LLM学生都应该在企业实习开展研究;

3. 两周一次组会, 按需1v1交流

给高年级学生自由, 为低年级学生提供必要的指引;

今天要回答的五个问题

五个模块递进，不是并列。

- | | | |
|----|--|--------|
| 01 | 模型在没见过的数据上为什么会错?
泛化 · 过拟合 · 正则化 | 35 min |
| 02 | 怎么把分类写成一个可优化的目标?
Softmax · 交叉熵 · 最大似然 | 35 min |
| 03 | 为什么非要深层非线性网络?
线性模型失效 · 隐藏层 · 通用近似 | 40 min |
| 04 | 有了目标函数，怎么把它降下去?
损失曲面 · SGD · 动量 · 自适应 | 50 min |
| 05 | 训得动之后，怎么训得好?
偏差方差 · Dropout · 归一化 | 45 min |

五个模块串成一条主线

问题

泛化

拟合能力 · 数据量 · 约束

目标

、交叉熵

从最大似然到损失函数

模型

、深度网络

隐藏层 · 激活 · 通用近似

算法

、优化

损失曲面 · SGD · Adam

调控

、正则化

偏差方差 · Dropout · 归一化

每个模块结束安排一组课堂练习，先自己判断，再对答案。

符号

术语第一次出现时给中英文，之后统一用英文。

$\mathbf{x} \in \mathbb{R}^D$

输入 / 特征向量，D 维

t 或 $\mathbf{t}$

真实标签；分类时为 one-hot

$\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N$

训练集，N 个样本

$\mathbf{w}, \boldsymbol{\theta}$

模型参数； $\boldsymbol{\theta}$ 泛指全部可学参数

$y(\mathbf{x}; \boldsymbol{\theta}), \hat{y}$

模型输出 / 预测类别概率

向量为列向量、粗体小写；矩阵粗体大写。下标 n 遍历样本， j / k 遍历维度或类别。

$L(\boldsymbol{\theta}), E(\mathbf{w})$

损失函数 / 误差函数

η

学习率 learning rate

λ

正则化强度

$B, \mathbb{B}$

批量大小 / 一个 minibatch

$\nabla_{\boldsymbol{\theta}} L$

损失对参数的梯度

01

泛化：学习到底难在哪

GENERALIZATION

本节要回答

01

训练误差降到 0，为什么反而是坏消息？

02

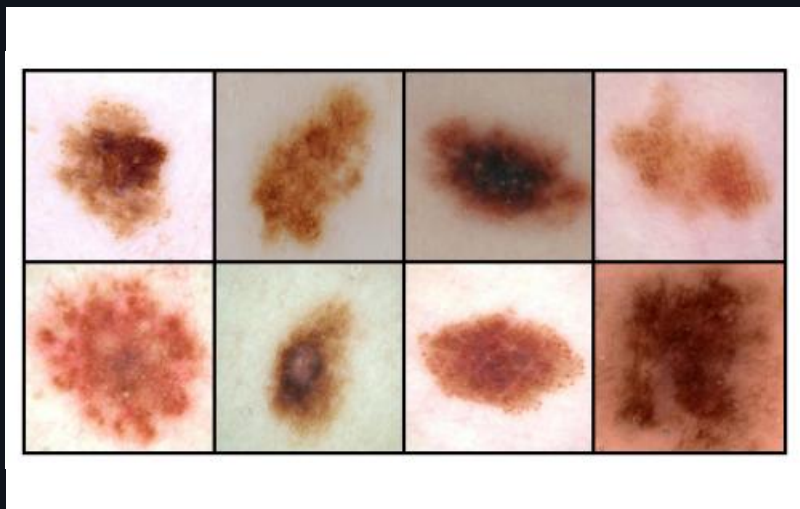
过拟合是模型的错，还是数据太少的错？

03

为什么必须同时留验证集和测试集？

先看两个已经被跨过的门槛

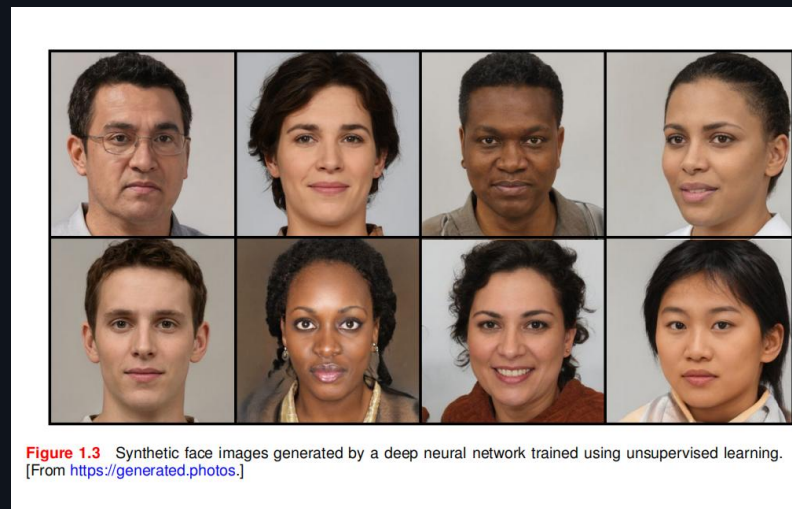
十年前都需要人类专家，现在由同一类模型完成。



判别 · 皮肤病变分类

12.9 万张临床图像，达到皮肤科医生水平

图 Bishop Fig 1.1；结果 Esteva et al., Nature 542 (2017)



生成 · 人脸合成

直接学习数据分布，图中人物均不存在

图 Bishop Fig 1.3，样本来自 generated.photos

共同的问题

只见过有限样本

却要在没见过的输入上做对

凭什么成立？

把「学习」写成一个可以分析的问题

问题设定

训练集 $\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$, i.i.d. 采自未知分布 $p(x, t)$

目标: 构造 $y(x; \theta)$, 在新的 x 上预测准

核心矛盾: 能测量的只有训练误差, 真正关心的是分布上的误差。两者之差 = 泛化误差

三个必须分清的对象

数据生成分布

$p(x, t)$

客观存在, 永远不可知

训练集

$\mathcal{D}$

有限样本, 唯一信息来源

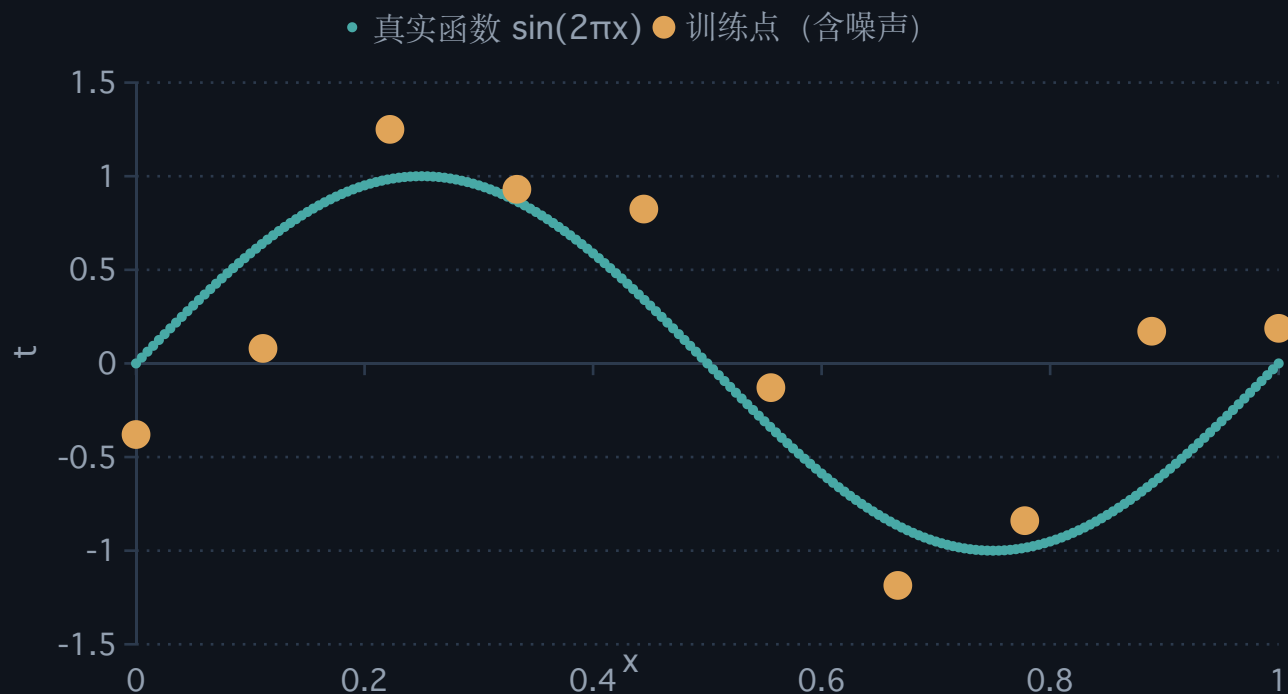
模型族

$\{y(x; \theta)\}$

自己划的范围, 决定能力上限

用一个最小实验把问题看清楚

$t = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.3^2)$ 。只给 10 个点。



为什么够用

真实规律已知，能直接判断对错

噪声不可约，是误差下界

维度低到能画出来，机理与百万参数网络一致

课程脚本现场拟合：训练集 $N=10$, $\sigma=0.3$ ；测试集 100 点独立采样。

模型族: M 阶多项式

$$y(x, w) = w_0 + w_1x + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

M 是超参数

人事先选定, 不由训练得到

后面的学习率、 λ 、批量大小同属此类

「线性模型」指对参数线性

对 x 非线性, 但对 w 线性

因此 $E(w)$ 是二次函数, 有唯一闭式解

模块 03 引入激活函数后, 这一性质消失

$M = 3$ 的模型族被 $M = 9$ 包含 —— 所以更大的 M 在训练集上永远不会更差

怎么在模型族里挑一个：最小化平方误差

$$E(w) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, w) - t_n\}^2$$

为什么有闭式解

E 是 w 的二次型 $\rightarrow$ 求导置零得正规方程

$$\Phi^T \Phi w^* = \Phi^T t$$

$\phi_{nj} = x_n^j$ 。解唯一，不需迭代。

本模块结论都在有闭式解的情形下得到，因此反映的是「学习」本身的困难，与「优化」无关。

深度网络两条都不满足

损失非凸；参数上百万

矩阵求逆代价 $O(d^3)$

$\rightarrow$ 模块 04 改用一阶迭代

同一批数据，四种阶数

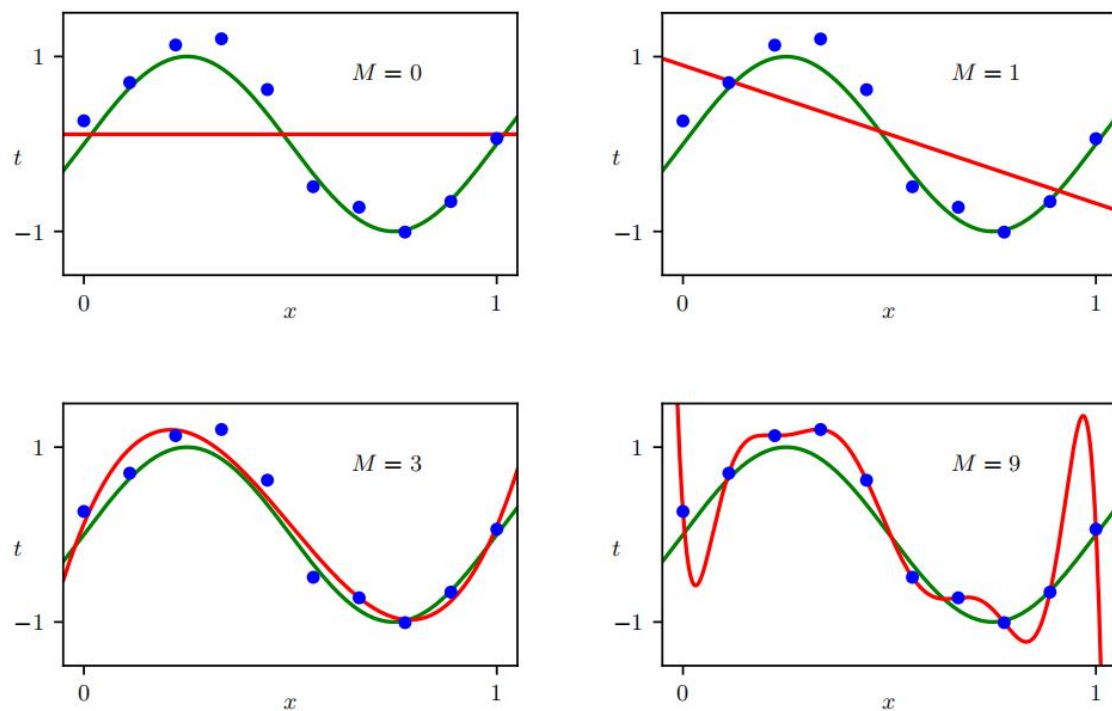


Figure 1.6 Plots of polynomials having various orders M , shown as red curves, fitted to the data set shown in Figure 1.4 by minimizing the error function (1.2).

图：Bishop & Bishop, Figure 1.6。绿线为真实函数，红线为拟合。

逐个看

$M = 0$

常数，完全没抓住结构

$M = 1$

直线，仍然欠拟合

$M = 3$

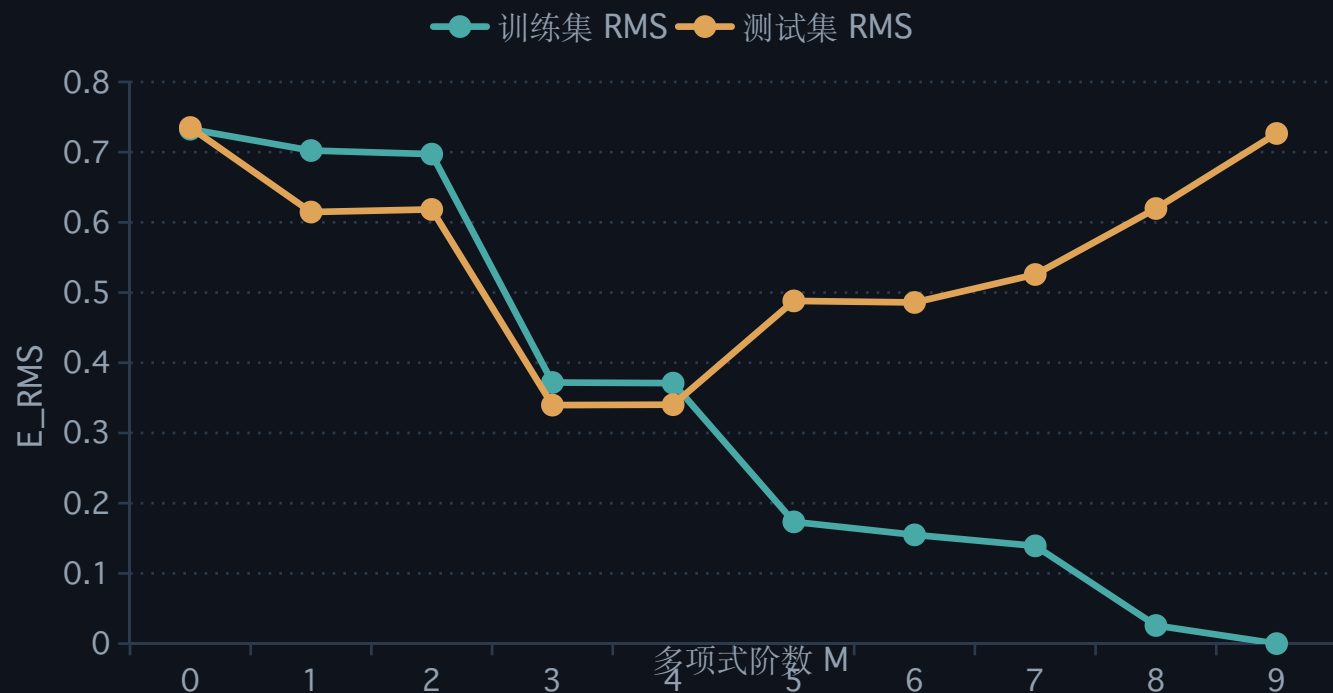
几乎重合于真实函数

$M = 9$

穿过每个点，点间剧烈震荡

$M = 9$ 训练误差 = 0

训练误差单调下降，测试误差是 U 型



课程脚本现场拟合：训练集 $N=10$, $\sigma=0.3$ ；测试集 100 点独立采样。

三个读数

01

训练 单调降， $M=9$ 处精确为 0

02

测试 最低点 $M=3$ (0.34)， $M=9$ 回到 0.73

03

两线分离处 = 过拟合起点

用训练误差挑模型，必然挑到最右端

过拟合的机理：系数在爆炸

同一批 10 个点，系数量级差了六个数量级。

	M = 0	M = 1	M = 3	M = 9
w0*	0.09	0.42	-0.57	-0.38
w1*	—	-0.65	14.21	-106.15
w3*	—	—	24.46	-1.8e+4
w9*	—	—	—	-1.9e+4

本课脚本拟合结果；空格表示该阶数下无此系数。现象与 Bishop Table 1.1 一致。

机理

穿过每个含噪点 → 点间必须剧烈翻折

翻折靠大系数相互抵消

抵消极不稳定：输入微变，预测大跳

病因是大系数 → 那就惩罚大系数

换个角度：过拟合也可能是数据太少

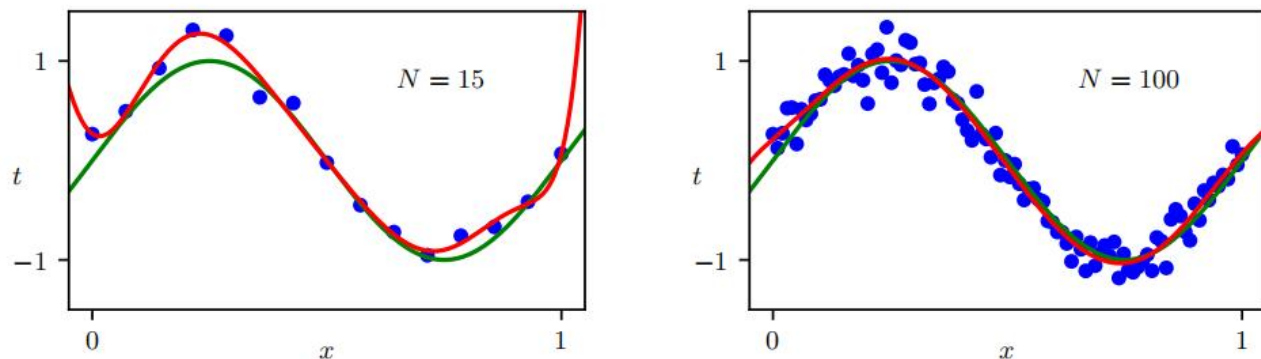


Figure 1.8 Plots of the solutions obtained by minimizing the sum-of-squares error function (1.2) using the $M = 9$ polynomial for $N = 15$ data points (left plot) and $N = 100$ data points (right plot). We see that increasing the size of the data set reduces the over-fitting problem.

图：Bishop & Bishop, Figure 1.8。两幅均为 $M=9$ 。

「 $M = 9$ 太复杂」不成立 —— 复杂度只有相对于数据量才有意义

同一个 $M=9$

$N = 15$: 边缘剧烈震荡

$N = 100$: 几乎完美还原 $\sin$

模型没有变化，只是数据变多

正则化：把「不要出现大系数」写进目标函数

$$\tilde{E}(w) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, w) - t_n\}^2 + \frac{\lambda}{2} \|w\|^2$$

解仍是闭式的

$$w^* = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T t$$

λI 顺带解决奇异问题 = 岭回归

λ 在做什么

权衡：拟合数据 $\leftrightarrow$ 保持系数小

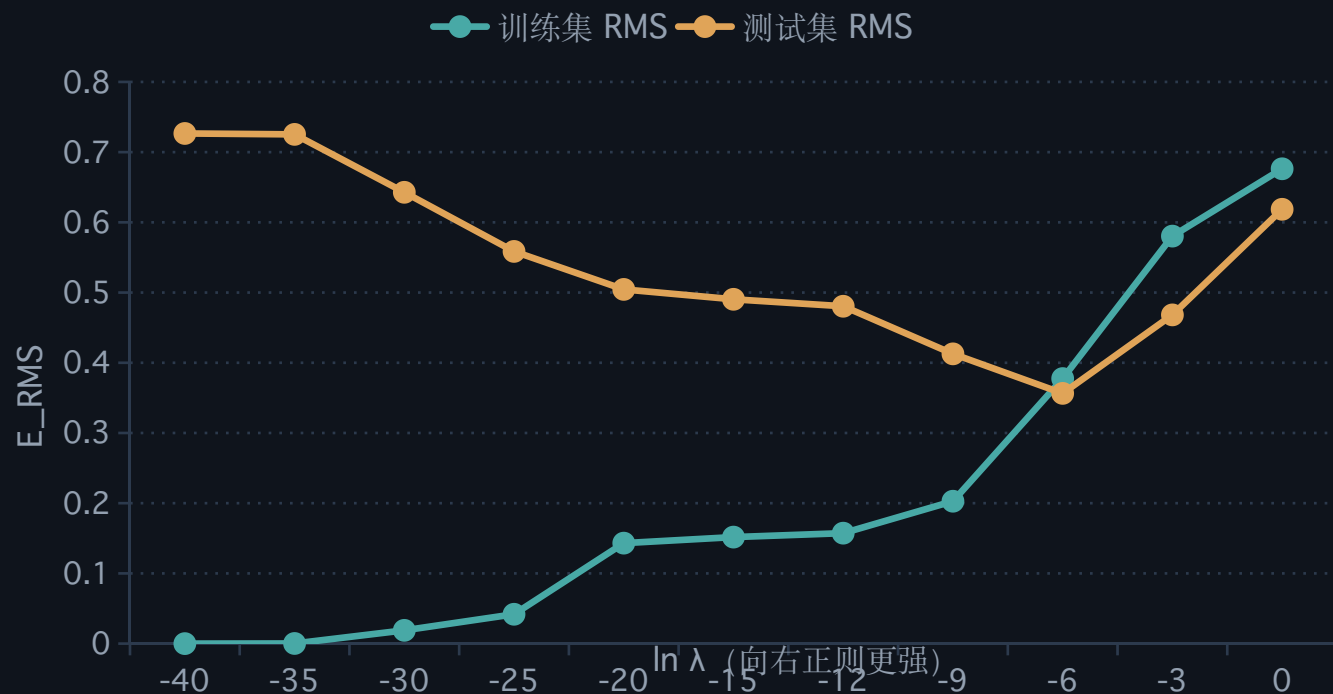
$\lambda \rightarrow 0$ ：退回最小二乘，系数自由爆炸

$\lambda \rightarrow \infty$ ：全部压到 0，退化成常数

λ 同样是超参数

用约束换泛化。模块 05 的 weight decay、Dropout、数据增强、Label Smoothing 都是它的不同实现。

λ 怎么选：同一条 U 型曲线又出现了



固定 $M=9$ ，只改 λ 。训练集 $N=10$ ， $\sigma=0.3$ ；测试集 100 点独立采样。

三个读数

$\ln \lambda = -40$: 训练 0，测试 0.73

$\ln \lambda \approx -6$: 测试 0.36，回到 $M=3$ 水平

$\ln \lambda = 0$: 正则过强，双双变差

同一个 9 阶模型，靠一个 λ 在过拟合与欠拟合间连续滑动

超参数要用哪份数据来选

M 和 λ 都不能用训练误差来选。

训练集

Training set

拟合参数 w

被模型直接优化

验证集

Validation set

挑超参数 M 、 λ 、结构

被人的选择间接优化

测试集

Test set

报告最终性能

全程只用一次

数据不够：K 折交叉验证

轮流留 1 份验证，取平均。代价：训练 K 次。

深度学习的常见做法

模型太大、训练太贵 → 直接留出固定比例（如 80/10/10）

深入：为什么验证集不能兼任测试集

学生最常出错的地方，也是很多论文结果无法复现的原因。

问题的实质

验证集也只是有限样本，其上的误差带随机波动

用它比较 50 个配置 = 对验证集做了一次优化，优化器是你自己

被选中者沾了「运气好」→ 验证误差是乐观估计

延伸：ImageNet、GLUE 这类公开基准的测试集被整个社区反复使用多年，社区层面同样存在这种过拟合 —— 这也是新基准不断被提出的原因之一。

一个可以现场做的验证

设 50 个配置真实性能完全相同，验证误差 = 相同均值 + 独立噪声

则「取 50 个误差结果的最小值」的期望必低于真实均值

候选越多，偏差越大 —— 与配置好坏无关

测试集必须在超参数全部锁定后才打开，且只打开一次。反复用它调参 = 把它变成第二个验证集。

模块 01 小结：三个旋钮

泛化不靠单一技巧，而是三个量的平衡。

1 模型容量

M、层数、宽度、参数量

调大 → 训练误差降，方差升

模块 03

2 数据量

N、数据增强

调大 → 几乎总有益，但成本高

模块 05

3 约束强度

λ 、Dropout、Early Stopping

调大 → 偏差升，方差降

模块 05

下一个问题：到现在只用了平方误差。分类该用什么损失，它从哪来？

练习 1：先自己判断，再讨论

Q1 训练误差有可能降到 0 吗？什么条件下会发生？

回到 $M=9$ 、 $N=10$ 那张图。参数量与样本量的关系？若有两个 x 相同、 t 不同的样本呢？

Q2 把 λ 也当参数、和 w 一起最小化，会发生什么？

写出 $\partial \tilde{E} / \partial \lambda$ 并看符号。这说明超参数与参数在优化上有什么本质区别？

Q3 你判断模型需要更多数据。怎么向老板证明？

别只说「数据不够」。用 25%、50%、100% 的数据分别训练，画测试误差随数据量的曲线，看是否仍在下降。

Q4 什么情况下才有理由期待模型能泛化？

回到问题设定，找出那条被默认成立、却几乎从未验证的假设。若测试数据来自另一分布，结论还剩多少？

每题先独立想 1 分钟，再与邻座对答案。

02

从最大似然到交叉熵

SOFTMAX & CROSS-ENTROPY

本节要回答

01

分类的输出是概率，怎么让模型产生合法的概率？

02

交叉熵损失是拍脑袋定的，还是推出来的？

03

为什么分类任务几乎从不用平方误差？

换一类任务：输出从实数变成类别

	回归 Regression	分类 Classification
输出空间	$t \in \mathbb{R}$ ，连续	$t \in \{1, \dots, K\}$ ，离散无序
模型输出	一个实数	K 类上的概率分布
合法性约束	无	每项 ≥ 0 ，总和为 1
典型损失	平方误差	交叉熵
典型任务	房价、生存期	垃圾邮件、图像分类、下一个词

「离散无序」是关键：把猫/鸡/狗编成 1/2/3，模型会以为「狗 - 猫 = 2」有意义。

标签编码：为什么必须用 one-hot

编码方式决定了损失函数会鼓励模型学到什么。

错误做法：整数编码

猫→1，鸡→2，狗→3

平方误差下：猫误判成鸡代价 1，成狗代价 4

模型被告知「猫和鸡更像」—— 这是编码凭空引入的

把标签看做分布，
交叉熵可以理解为「两个分布有多不一样」的自然度量。

正确做法：one-hot

猫 (1,0,0)，鸡 (0,1,0)，狗 (0,0,1)

任意两类距离相等，不引入顺序假设

更关键的一层：one-hot 本身就是一个概率分布。标签与模型输出成为同类对象，可以直接用衡量分布差异的工具比较。

先搭最简单的模型：一层线性变换

$$z = Wx + b, W \in \mathbb{R}^{K \times D}, b \in \mathbb{R}^K$$

W 权重 b 偏置

z : **logits**, 未归一化得分

代码里叫全连接层 / linear layer

参数量

$$K \times D + K$$

$$28 \times 28 \text{ 十分类: } 784 \times 10 + 10 = 7850$$

每行 w_k 是第 k 类的模板, $w_k^T x$ 是匹配度

问题

z_k 可正可负、可以是 -3.7 或 $+128$

但我们需要一个概率分布

直接归一化 $z_k / \sum_j z_j$ 行不行?

不行：分母可能为 0，负的 z_k 给出负概率。需要先把实数压成正数、再归一化的映射。

Softmax: 把 logits 变成合法的概率分布

$$\hat{y}_k = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}$$

01 非负

$\exp(\cdot)$

满足概率第一条

名字的来历

$\arg \max$ 的平滑近似。「soft」指可微，不是结果柔和。

02 归一

分母是全部分子之和 $\Rightarrow \sum_k \hat{y}_k = 1$

满足第二条

03 保序

$\exp$ 严格单调 $\Rightarrow z_i z_j \Rightarrow \hat{y}_i \hat{y}_j$

预测结果不会变化

一个反直觉的性质

$\exp$ 放大差距: $(1,2,3) \rightarrow (0.09,0.24,0.67)$,

logit 差 1 换来概率差数倍

深入: Softmax 的起源

这个形式在统计物理里比机器学习早了近一百年。

Boltzmann 分布 (1870s)

$$p_k = \frac{\exp(-E_k/kT)}{\sum_j \exp(-E_j/kT)}$$

把 z_k 看作负能量, 两式完全相同

Gibbs (1902). Elementary Principles in Statistical Mechanics.

为什么偏偏是指数

最大熵原理: 只知道各类得分期望时, 熵最大的分布必为指数族

即 Softmax 是假设最少的那个分布, 没偷偷引入额外结构

这解释了它为何在物理、统计、机器学习里被反复发现

温度: $\text{softmax}(z/T)$ 。 $T \rightarrow 0$ 趋于 one-hot, $T \rightarrow \infty$ 趋于均匀。大模型采样的 temperature 就是它。

深入：平移不变性，以及一行代码的救命作用

平移不变性

$$\text{softmax}(z + c1) = \text{softmax}(z)$$

分子分母同乘 $\exp(c)$ ，约掉

推论：Softmax 的解不唯一， b 有一个自由度是冗余的。

工程实践问题

$z_k = 1000 \rightarrow \exp$ 溢出为 $\infty \rightarrow \text{NaN}$

全部 $z_k = -1000 \rightarrow$ 分子分母同时下溢 $\rightarrow 0/0$

标准修正

$$\hat{y}_k = \frac{\exp(z_k - z_{\max})}{\sum_j \exp(z_j - z_{\max})}$$

由平移不变性，结果数学上完全相同；最大指数变成 1，不会上溢

配套用 log-sum-exp 计算 $\log \sum_j \exp(z_j)$

PyTorch 的 `CrossEntropyLoss` 内部已实现

向量化：为什么不能写成 for 循环

逐样本 vs 批量

```
for x in batch:
```

$$z = W @ x + b$$

数学没错，GPU 计算非常慢

$$Z = XW^T + b^T$$

$X \in \mathbb{R}^{B \times D}$ ：B 个样本按行堆叠

$Z \in \mathbb{R}^{B \times K}$ ：全部 logits

b 广播到每一行

Softmax 沿类别维归一化，不是样本维

为什么快这么多

单次大矩阵乘能填满数千个 GPU 核心

B 次小矩阵向量乘，时间大多花在启动开销与显存搬运

量级差别是一到两个数量级

批量最初是工程决定，后来才发现对优化也有好处

常见错误：归一化维度写错，把不同样本的得分混在一起归一。运行不报错，只是永远学不好。

损失函数从哪来：写出似然，取负对数

01 单样本概率

真实类别的概率即 $\hat{y}_{n,t_n}$ ，用 one-hot 写成 $\prod_k \hat{y}_{nk}^{t_{nk}}$

02 整个数据集

i.i.d. $\Rightarrow p(T|X, \theta) = \prod_n \prod_k \hat{y}_{nk}^{t_{nk}}$

03 取负对数

连乘会下溢且难求导；取对数变连加，取负号把最大化变成最小化

04 得到交叉熵

$$L(\theta) = - \sum_n \sum_k t_{nk} \log \hat{y}_{nk}$$

交叉熵不是被规定的，是「让模型给训练数据分配最大概率」的直接结果。

深入：熵、交叉熵、KL 散度的关系

熵

$$H(p) = - \sum_k p_k \log p_k$$

分布自身的不确定性

交叉熵

$$H(p, q) = - \sum_k p_k \log q_k$$

用 q 编码来自 p 的信息，平均代价

KL 散度

$$KL(p||q) = \sum_k p_k \log \frac{p_k}{q_k}$$

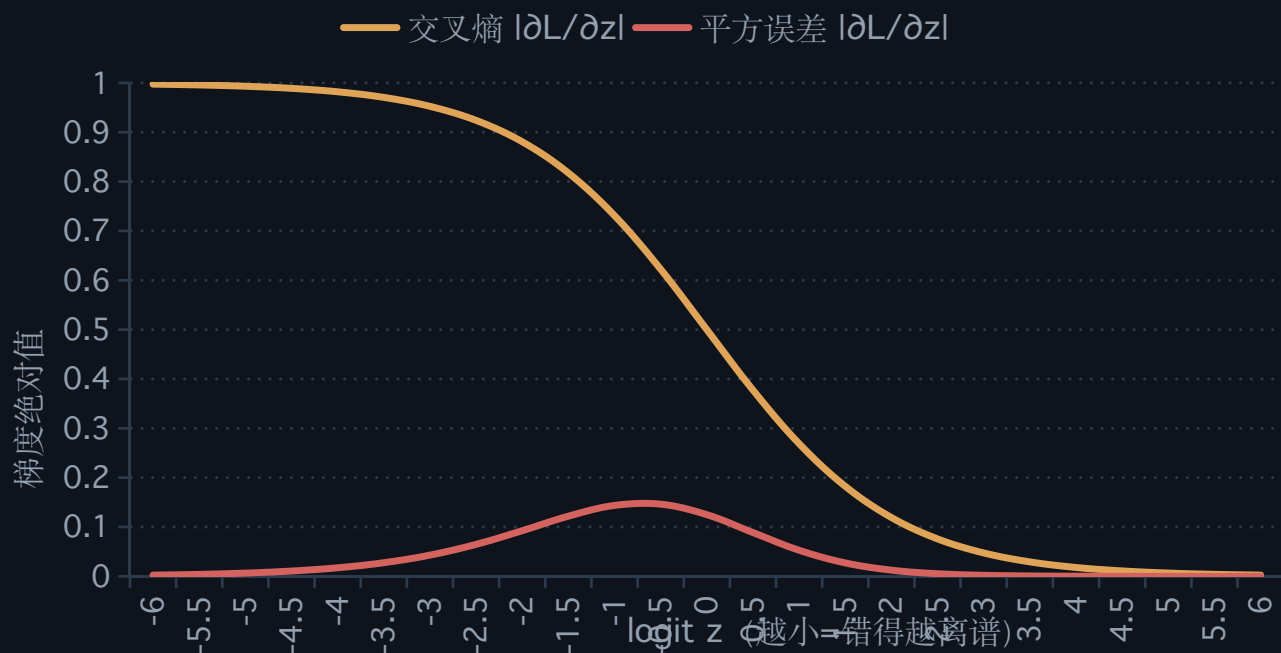
这份代价里多付出的部分

$$H(p, q) = H(p) + KL(p||q)$$

分类时 p 是 one-hot $\Rightarrow H(p) = 0 \Rightarrow$ 交叉熵 = KL 散度。最小化交叉熵就是把模型分布往真实标签分布上推； $H(p)$ 是常数，优化时可以不管。

深入：分类为什么不用平方误差

二分类、 $t = 1$ ，比较两种损失对 $\text{logit } z$ 的梯度大小。



解析式：交叉熵 $1 - \hat{y}$ ；平方误差 $(1 - \hat{y})\hat{y}(1 - \hat{y})$ 。

$z = -6$ (几乎完全判错)：交叉熵 **0.998**，平方误差 **0.0025**，差约 400 倍

原因：链式法则多出 $\sigma'(z) = \hat{y}(1 - \hat{y})$ ，两端饱和趋零

结论：错得越离谱，平方误差越学不动

交叉熵的 $\log$ 恰好抵消 Softmax 的 $\exp$ ，饱和被消除。

推导: Softmax + 交叉熵的梯度出奇地简洁

01 $\log \hat{y}_k = z_k - \log \sum_j \exp(z_j)$

对 Softmax 取对数, 指数被消掉

02 $L = - \sum_k t_k z_k + \log \sum_j \exp(z_j)$

代入交叉熵, 用 $\sum_k t_k = 1$

03 $\frac{\partial}{\partial z_i} \log \sum_j \exp(z_j) = \hat{y}_i$

log-sum-exp 的导数正是 Softmax 自身

04 $\frac{\partial L}{\partial z_i} = \hat{y}_i - t_i$

两项合并

向量形式 $\nabla_z L = \hat{\mathbf{y}} - \mathbf{t}$: 预测概率减真实标签。多了往下压, 少了往上提, 比例正好等于错的量。

深入：这个形式为什么重要

没有饱和项

不含 σ' 这类趋零因子。无论初始化多差，第一步都有有效梯度。

误差直接反传

$\hat{y} - t$ 是输出层误差信号，反向传播就是把它逐层往回送。

对参数的梯度也简洁

$\nabla_w L = (\hat{y} - t)x^T$ ：误差与输入的外积，一行实现。

框架把两者合并

正因 $\log$ 与 $\exp$ 要抵消，PyTorch 的 `CrossEntropyLoss` 直接接收 logits。

损失函数的选择，最终是在选择梯度的形状。

特例检查：K=2 时它退化成什么

把 K=2 代进 Softmax

$$\hat{y}_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \sigma(z_1 - z_2)$$

只有差值起作用 —— 平移不变性的又一次体现

两类问题只需一个输出，而不是两个

多标签任务（一张图同时是「猫」和「室内」）不能用 Softmax —— 它强制概率和为 1。应对每类独立用 sigmoid + BCE。

交叉熵同步退化

$$L = -[t \log \hat{y} + (1 - t) \log (1 - \hat{y})]$$

即二元交叉熵 BCE，也就是 logistic 回归的损失

梯度依然是 $\hat{y} - t$

logistic 回归是本框架的 $K = 2$ 特例

PyTorch: 多分类 CrossEntropyLoss（接收 logits + 整数标签），多标签 BCEWithLogitsLoss。两者都内部做 log-sum-exp。

模块 02 小结

01 one-hot 把标签变成分布

从此能用衡量分布差异的工具定义损失

02 Softmax 把 logits 变成分布

非负、归一、保序；本质是最大熵下的指数族

03 交叉熵由最大似然推出

one-hot 情形下等于 KL 散度，不是人为规定

04 梯度是 $\hat{y} - t$

无饱和、可直接反传；这就是不用平方误差的原因

但模型仍只是 $z = Wx + b$ 一层线性变换。它到底能表达什么、不能表达什么？

练习 2: Softmax 与交叉熵

Q1 把所有 logits 乘以 10，预测类别会变吗？置信度会变吗？

分别考察保序性与温度参数。这解释了模型为何可能「非常自信地答错」——它与模块 05 的 Label Smoothing 有什么关系？

Q2 证明：对 one-hot 标签，最小化交叉熵等价于最小化 KL 散度。

写出 $H(p, q) = H(p) + KL(p||q)$ ，再说明 one-hot 的 $H(p)$ 等于多少。约定 $0 \cdot \log 0 = 0$ 。

Q3 训练时 loss 突然变成 NaN，从哪几处查起？

至少三条：手写 $\log(\text{softmax}(z))$ 、标签越界、学习率过大致 logits 爆炸。哪一条可以靠换用框架自带损失直接消除？

Q4 一张图可同时属于「猫」和「室内」。还能用 Softmax 吗？

Softmax 强制 $\sum \hat{y} = 1$ ，这在多标签场景意味着什么？给出替代方案并写出损失形式。

03

为什么必须是深度非线性网络

DEEP NEURAL NETWORKS

本节要回答

01

线性模型到底哪里不够用？

02

为什么堆两层线性层没有带来非线性？

03

既然一个隐藏层就足够，为什么还要做深？

线性模型暗含两个假设

$z = Wx + b$ 看起来什么都没假设，其实约束非常强。

线性性 Linearity

x_j 增一单位， z_k 变化恒为 w_{kj} ，与当前取值、其他特征都无关

排除一切交互作用与饱和效应

单调性 Monotonicity

x_j 增大时 z_k 只能一直增或一直减，不能先增后减

排除一切「过犹不及」型规律

反例一

收入 → 还款概率

违反线性性，仍单调

反例二

体温 → 健康状况

连单调性也违反

反例三

像素 → 图像类别

特征本身没有固定语义

反例一：违反线性性，但固定基函数能救

现象

收入越高越可能还款 —— 单调性成立

但边际效应递减：3 千→6 千提升大，30 万→30.3 万几乎无变化

修补办法

先做固定变换 $\varphi(x) = \log x$ ，再对 $\varphi(x)$ 做线性模型

这类人工设计的 φ 叫**固定基函数**；模块 01 的 $\varphi_j(x) = x^j$ 即其特例

为什么这仍算「线性模型」

$y = w^{\text{T}}\varphi(x) + b$ 对 w 依然线性

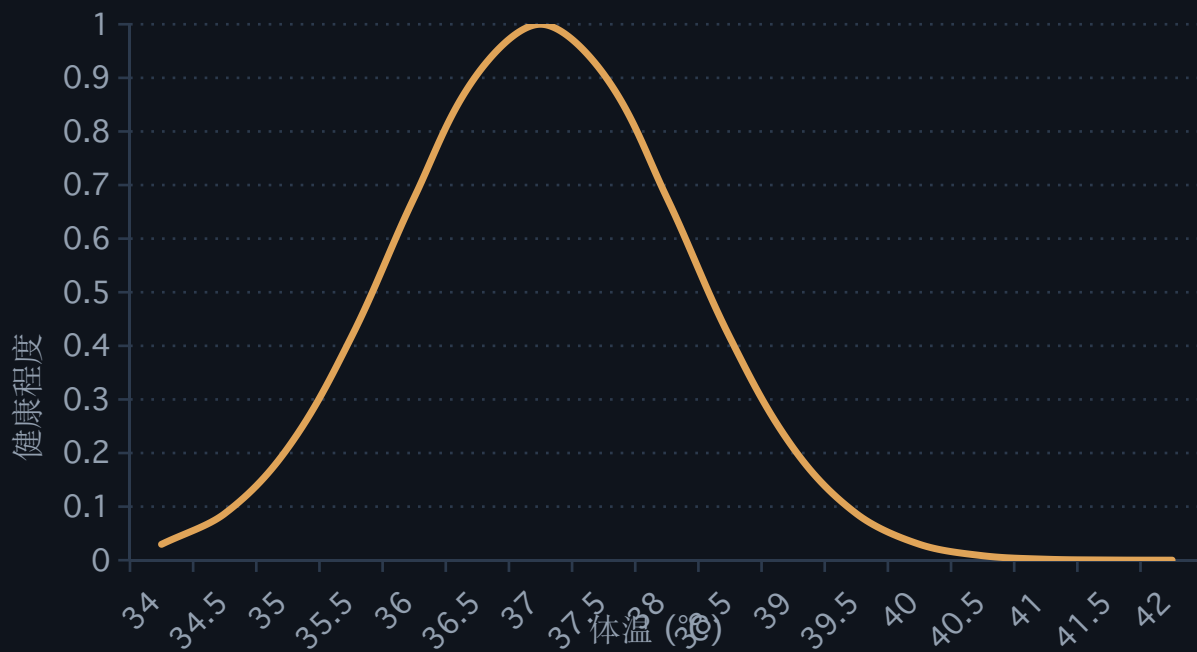
闭式解、凸性全部保留

非线性被塞进 φ ，而 φ 由人指定、不参与训练

这就是核方法与传统特征工程的全部思路：模型很简单，智慧都在 φ 的设计里。在低维、领域知识充分的问题上非常有效 —— 直到遇到下面两个反例。

反例二：连单调性都不成立

根据体温预测健康：37°C 最健康，往两边都变差。



示意曲线，说明形状，非真实医学数据。

单调性一破，麻烦就大了

线性模型只能给出「越高越健康」或「越低越健康」，两者都明显错误

$\log$ 、 $\sqrt{\cdot}$ 也救不了 —— 单调复合单调仍是单调

能救的办法是换基函数 $\varphi(x) = (x - 37)^2$ 。但这要求我们事先知道峰值在 37°C —— 真实问题里，正是这个「事先知道」最难做到。

反例三：特征根本没有固定语义

图像分类的困境

「第 (100,150) 个像素较亮」对判断是不是猫没有固定含义

平移几像素、换光照、换角度 → 全部像素变，类别不变

有意义的是「边」「猫耳轮廓」—— 像素间高度非线性的组合

手工设计也不可行：SIFT、HOG 是几十年研究成果，但换个任务就要重来，且天花板明显。

固定基函数为什么彻底失效

每维划 10 格 $\Rightarrow D$ 维有 10^D 个格子

$D = 784$ 的小图像已是 10^{784} ，远超宇宙原子数

这就是维度诅咒 (curse of dimensionality)

出路只剩一个：不要人来设计 ϕ ，让它从数据里学出来。这一句就是深度学习与传统机器学习的分界线。

让基函数可学：加一个隐藏层

把 φ 换成带参数、能被梯度更新的形式： $\sigma(W^{(1)}x + b^{(1)})$ 。

输入层

$$x \in \mathbb{R}^D$$

原始特征

隐藏层

$$h = \sigma(W^{(1)}x + b^{(1)})$$

学出来的表示 representation

输出层

$$z = W^{(2)}h + b^{(2)}$$

在新表示上做线性分类

变化

$W^{(1)}$ 也被优化 $\rightarrow$ 基函数为当前任务量身学出

这就是多层感知机 MLP

代价

对参数不再线性 $\rightarrow$ 闭式解消失、损失非凸

从此只能迭代优化 —— 模块 04 的理由

关键：如果去掉 σ ，深度就毫无意义

假设隐藏层不加激活函数，直接把两个线性层接起来：

$$01 \quad h = W^{(1)}x + b^{(1)}$$

第一层

$$02 \quad z = W^{(2)}h + b^{(2)}$$

第二层

$$03 \quad z = W^{(2)}W^{(1)}x + (W^{(2)}b^{(1)} + b^{(2)})$$

代入展开

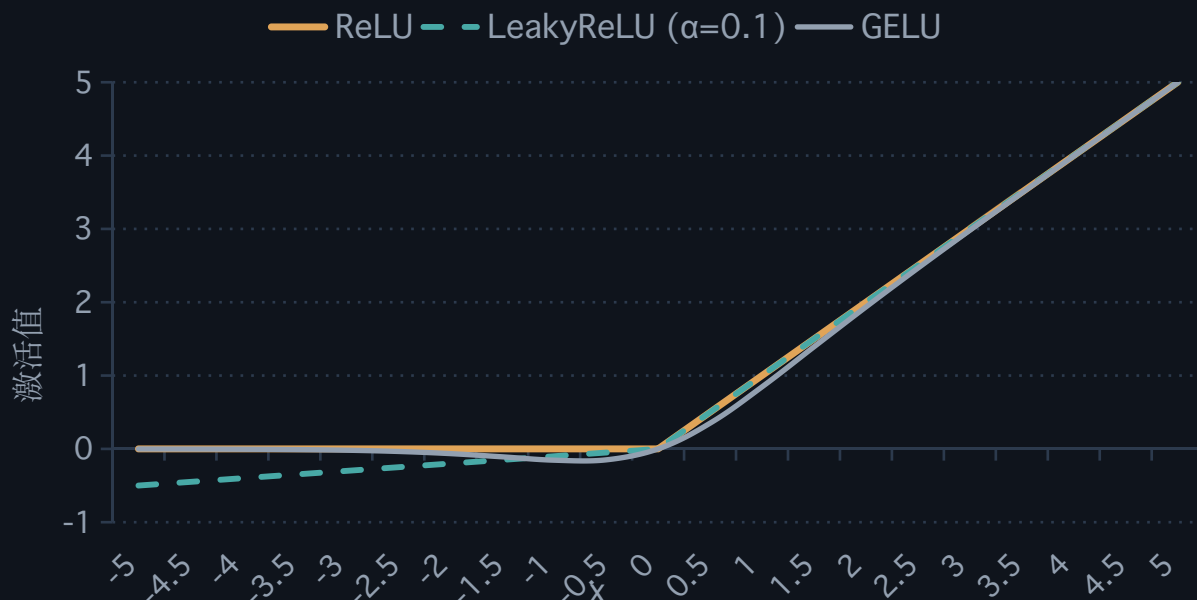
$$04 \quad z = \tilde{W}x + \tilde{b}$$

令 $\tilde{W} = W^{(2)}W^{(1)}$

两层线性层 = 一层线性层。堆再多也一样：线性变换的复合仍是线性变换。

非线性激活是深度网络成立的前提。

现在的默认选择: ReLU 及其变体



GELU 用 sigmoid 近似 $x \sigma(1.702x)$ 绘制。

ReLU 为什么成默认

$\max(0, x)$, 正半轴导数恒为 1 —— 梯度不衰减, 这是最重要的性质

计算只需一次比较; 输出天然稀疏

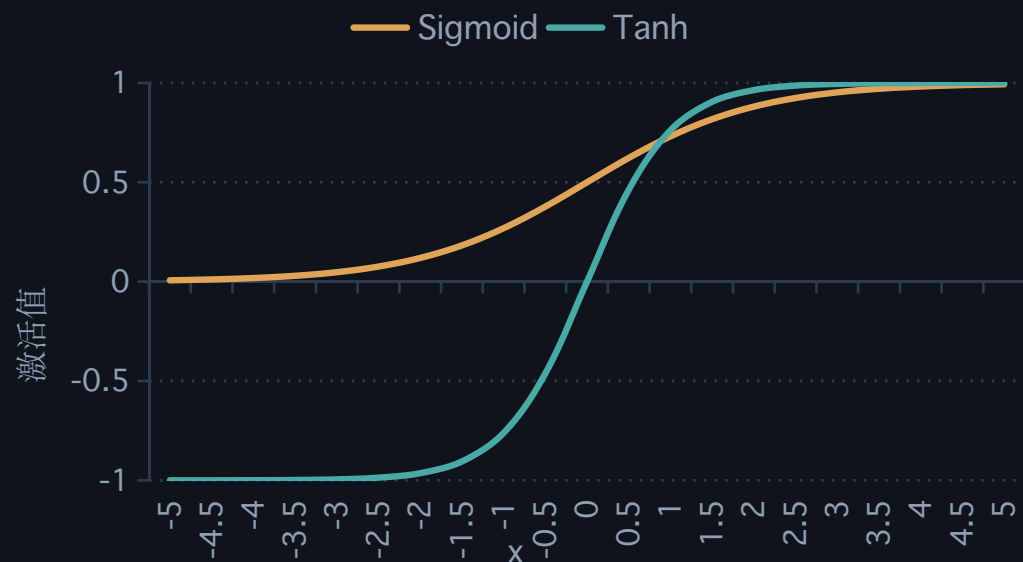
它的问题: 死神经元

输入长期为负 $\rightarrow$ 梯度恒 0 $\rightarrow$ 参数永不更新

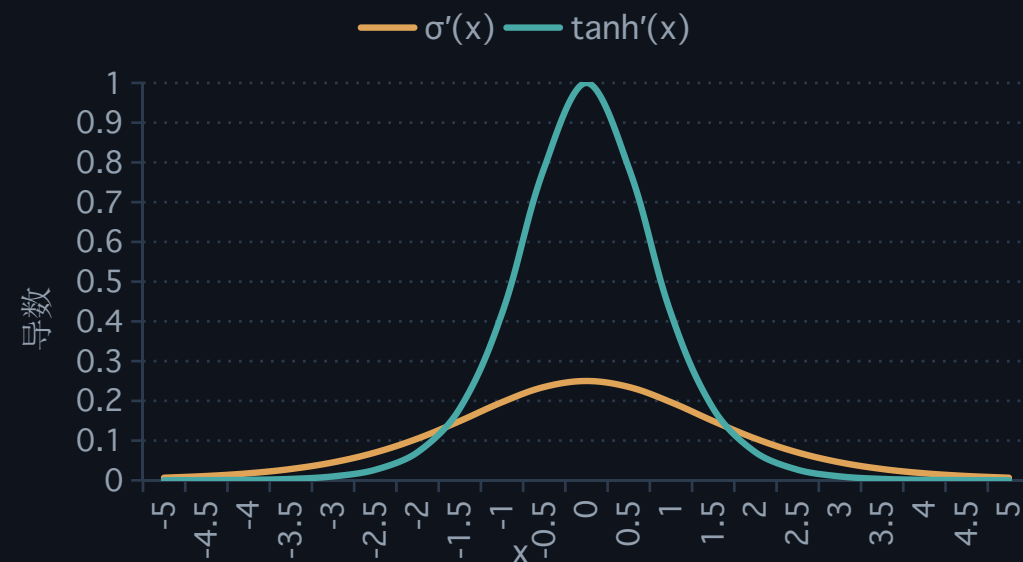
LeakyReLU 保留负半轴小斜率, GELU 用光滑过渡

实务: 先用 ReLU; Transformer 类用 GELU 或 SwiGLU。

饱和型激活：Sigmoid 与 Tanh，以及它们的导数



左：函数本身。Sigmoid $\rightarrow (0,1)$, Tanh $\rightarrow (-1,1)$ 。



右：导数。 σ' 峰值仅 0.25, $\tanh'$ 峰值为 1。

Tanh 优于 Sigmoid: 输出零中心, 梯度不全同号, 收敛更快 (LeCun 1998)

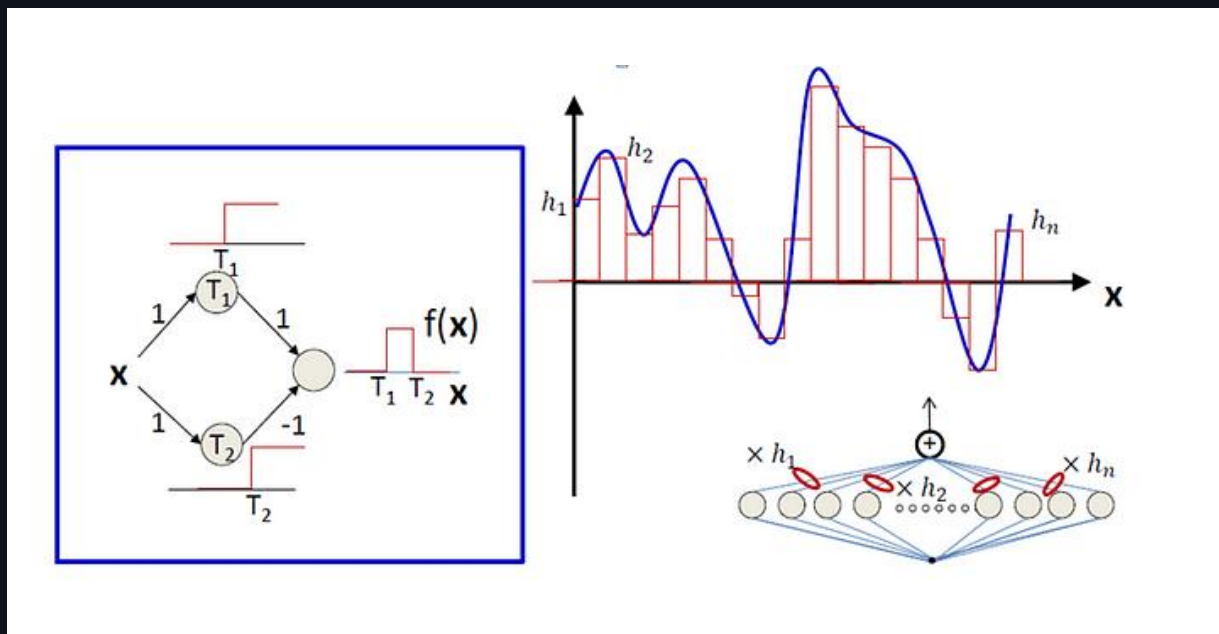
共同的病: 导数几乎为 0, 进入饱和区就学不动

三类激活函数：怎么选

激活函数	形式	优点	缺点	什么时候用
ReLU	$\max(0, x)$	正半轴梯度恒 1，最快，稀疏	死神经元；非零中心	默认 (CNN、MLP)
LeakyReLU / ELU	负半轴留小斜率	缓解死神经元	多一个超参数 α	大量神经元失活时
GELU / SiLU	光滑、非单调	梯度更平滑，深层更好	计算稍贵	Transformer 类
Tanh	零中心， $(-1,1)$	比 Sigmoid 收敛快	仍然饱和	RNN、需有界输出
Sigmoid	$(0,1)$	可解释为概率	梯度最大仅 0.25	只用在输出层

激活函数很少是瓶颈。先把 ReLU 跑通，再回头调它。

通用近似定理：一个隐藏层就够了



构造性直观：阶跃函数拼台阶，叠加逼近任意连续函数。

定理陈述

隐藏层神经元足够多时，带「挤压」激活的单隐藏层前馈网络可在紧致集上以任意精度逼近任意连续函数

Cybenko (1989). MCSS 2(4), 303–314.

非线性网络的表达能力有没有上限？

答案是没有。这为整个领域提供了合法性基础。

深入：通用近似定理没说的三件事

01 存在性 $\neq$ 构造性

保证「存在」一组参数，完全没说怎么找到。梯度下降能否找到是另一个问题。

02 「足够多」可能是天文数字

所需神经元数可随精度与输入维度指数增长。理论可行，工程不可行。

03 训练逼近好不代表泛化好

讲的是在训练数据上逼近得多好。而训练集上逼近得越完美，越可能过拟合。

那为什么做深而不是做宽

同样的函数，深层网络往往只需**指数级更少**的神经元。深度带来复合：边 $\rightarrow$ 纹理 $\rightarrow$ 部件 $\rightarrow$ 物体；宽度只能并列增加模板数量，深度却能重复利用它们。

怎么求梯度：反向传播就是链式法则

网络是函数复合，梯度按复合顺序反着乘回去。

01	输出层误差	$\delta^{(2)} = \hat{y} - t$	模块 02 的结论直接拿来用
02	输出层参数	$\nabla_{W^{(2)}} L = \delta^{(2)} h^{!T}$	误差与该层输入的外积
03	误差回传一层	$\delta^{(1)} = (W^{(2)T} \delta^{(2)}) \odot \sigma'(a^{(1)})$	按权重转置回传，再逐元素乘激活导数
04	隐藏层参数	$\nabla_{W^{(1)}} L = \delta^{(1)} x^{!T}$	形式与输出层完全一致

注意第 3 步的 σ' ：每回传一层就额外乘一次激活导数。下一页所有问题都出在它身上。

梯度消失与梯度爆炸：连乘的必然后果

L 层网络的梯度含 L 个因子连乘：略小于 1 就指数衰减，略大于 1 就指数放大。

层数 L	Sigmoid $(0.25)^L$	ReLU $(1)^L$
1	2.5e-1	1
3	1.6e-2	1
6	2.4e-4	1
10	9.5e-7	1

按 σ' 的最大值 0.25 计算，已是最乐观估计。

梯度消失

10 层 Sigmoid 网络浅层梯度约 10^{-6} → 浅层几乎不更新，退化
成只有最后几层在学

梯度爆炸

因子 > 1 时指数增长，参数跳出合理区域，loss 变 NaN。RNN
上尤其常见。

对策（模块 04 / 05 展开）

消失：ReLU 类激活 · 残差连接 · BN · 合适初始化

爆炸：梯度裁剪 · 更小学习率 · Xavier / He

模块 03 小结

01 线性模型的边界

线性性与单调性两条假设，在图像上彻底失效

02 固定基函数不够

维度诅咒使人工设计 ϕ 在高维不可行

03 隐藏层让 ϕ 可学

代价是失去闭式解与凸性

04 非线性是前提

去掉激活函数，任意深度都塌缩成一层

05 通用近似 $\neq$ 可学习

只保证存在解，不保证找得到、也不谈泛化

06 深度的代价

连乘导致梯度消失或爆炸

模型有了、损失有了、梯度也会求了。下一个模块：怎么真的把损失降下去。

练习 3：深度与非线性

Q1 把所有权重初始化为 0 会怎样？初始化为同一非零常数呢？

写出同层两个神经元的梯度表达式并比较。这个现象叫「对称性」。

Q2 设计实验，验证两层 ReLU 网络可以逼近任意函数。

自定非单调函数（如 $\sin(3x) \cdot e^{(-x^2)}$ ），在 $[-3, 3]$ 采样生成训练/测试集，用两层 ReLU 拟合，观察隐藏层宽度从 4 增到 256 的变化。

Q3 为什么优先加深而不是加宽？两者在参数量与表达能力上如何权衡？

从「复合 vs 并列」的角度想。再想反面：深度带来的代价是什么？

Q4 只用 ReLU 就一定不会梯度消失吗？

ReLU 正半轴导数为 1，但反传因子还有权重矩阵。若权重普遍很小会怎样？死神经元又是哪一种「消失」？

Q3 是课后编程题，30 行以内可完成。Q1、Q4 现场讨论。

04

把损失真的降下去

OPTIMIZATION

本节要回答

01

我们优化的目标，和真正关心的目标是同一个吗？

02

深度网络的损失曲面上，最大的敌人是局部极小吗？

03

从 SGD 到 Adam，每一步各解决了什么问题？

先分清： 我们想优化什么， 实际优化了什么

期望风险（真正想要的）

$$\mathbb{R}(\theta) = \mathbb{E}_{(x,t) \sim p}[\ell(y(x; \theta), t)]$$

p 未知，输入无法穷举 $\rightarrow$ 永远算不出来

经验风险（实际优化的）

$$\mathbb{R}_{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \ell(y(x_n; \theta), t_n)$$

可精确计算、可求梯度。最小化它 = 经验风险最小化 ERM

凭什么可以替代

i.i.d. 下经验风险是无偏估计: $\mathbb{E}_{\mathcal{D}}[\mathbb{R}_{emp}] = \mathbb{R}$

大数定律: $N \rightarrow \infty$ 时收敛

但无偏只对固定的 θ 成立。用同一批数据去挑 θ ，保证立刻失效 —— 这是模块 01 过拟合的严格来源。

两者之差 $\mathbb{R} - \mathbb{R}_{emp}$ = 泛化误差。模块 04 负责降 $\mathbb{R}_{emp}$ ，模块 05 负责控这个差。

优化在哪里停下来：三种临界点

梯度下降在 $\nabla L = 0$ 处停止。但梯度为零有三种情况。

全局最小

Global minimum

Hessian 特征值全正

真正的最低点。非凸问题里无法确认自己在此。

局部最小

Local minimum

Hessian 特征值全 正

邻域内最低。经典教材认为这是主要障碍。

鞍点

Saddle point

特征值有正有负

某些方向升、某些方向降。梯度为零但不是极值。

判别：看 Hessian $\nabla^2 L$ 特征值符号。在深度网络这种极高维空间里，这三种情况出现的概率极不均衡。

深入：高维空间里，真正常见的是鞍点

一个粗略但有力的计数论证

临界点处 Hessian 有 d 个特征值

若符号独立、各半，「全正」概率约 2^{-d}

$d = 10^6$ 时该概率实际为零

随机遇到的临界点几乎必然是鞍点。「陷进局部极小出不来」
这个担忧，在深度网络里基本不成立。

这个论证要打折扣的地方

特征值符号并不独立，损失曲面也不随机。这是启发式，不是定理。

更严格的结论来自随机矩阵理论与实证：损失越低的临界点，负特征值比例越小。

Dauphin et al. (2014), NeurIPS.

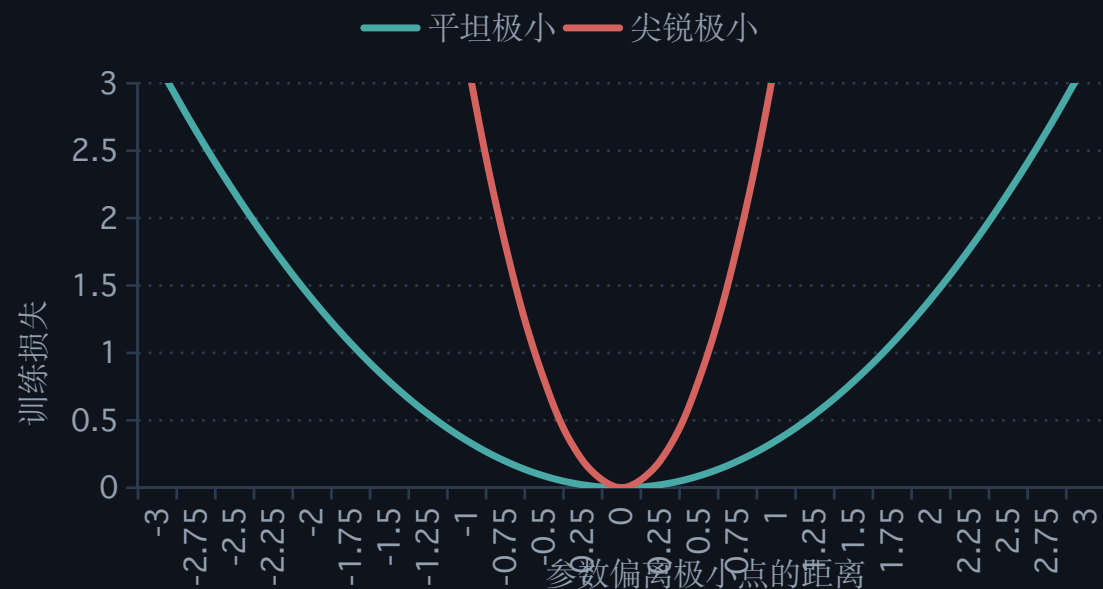
实践含义

鞍点的麻烦不是「出不去」，而是附近梯度极小、走得极慢（平台区）

SGD 噪声与动量法之所以有效，很大一部分原因就是帮助快速逃离平台

不是所有极小值都一样好：平坦 vs 尖锐

两个极小值的训练损失可以完全相同，泛化能力却差别很大。



示意图：中心处损失相同，曲率不同。

为什么偏好平坦极小

训练曲面与测试曲面之间有一个小平移（训练集只是有限样本）

尖锐极小处：平移让损失急剧上升

平坦极小处：损失几乎不变 → 更鲁棒

小批量 SGD 的噪声倾向于找到平坦解；
大批量训练容易掉进尖锐极小，需要额外补偿。

真实的损失曲面长什么样

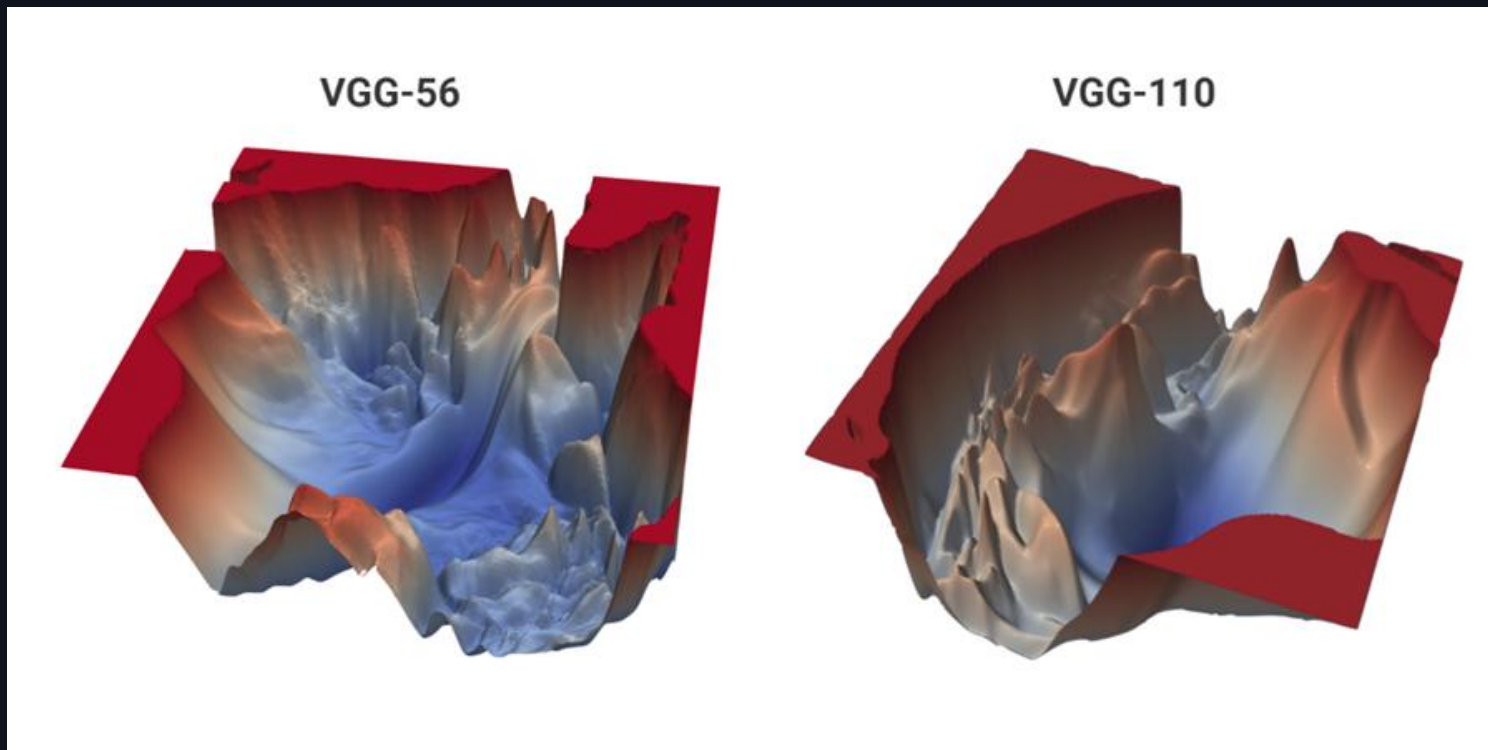


图: Li, H. et al. (2018). Visualizing the Loss Landscape of Neural Nets. NeurIPS.

两张都是无残差连接的 VGG，右侧更深

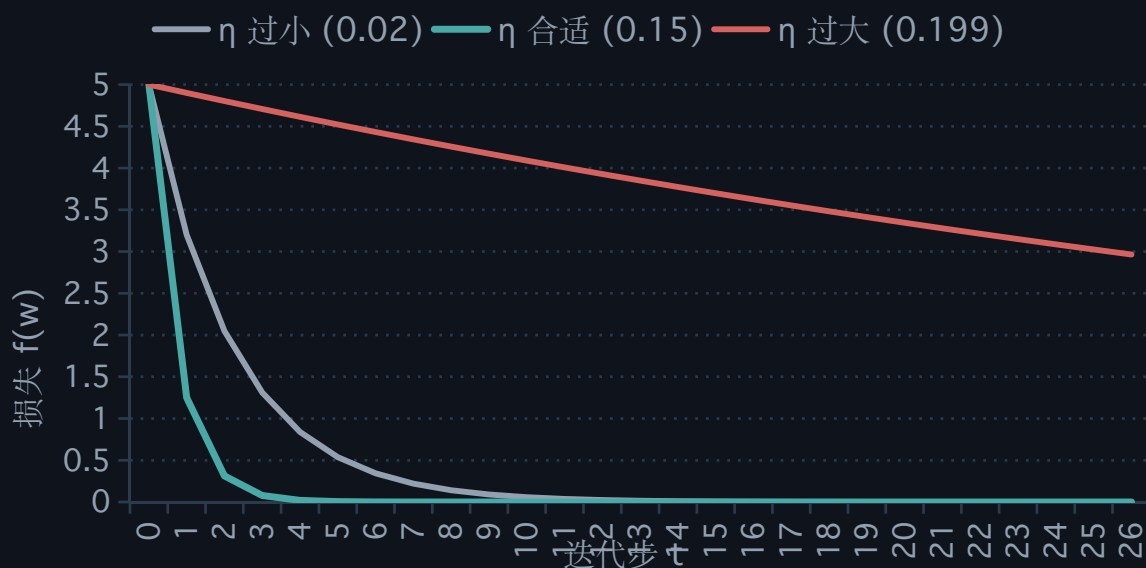
56 $\rightarrow$ 110 层，曲面明显更崎岖，中心平坦盆地几乎消失

对应「深层网络更难训练」这一经验事实

原论文另一组对照：加上残差连接后，同样深度的曲面重新变平坦 —— 这是 ResNet 有效的一个直观解释。

梯度下降：唯一的动作是沿负梯度走一步

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$



真实运行: $f(w) = \frac{12}{a} w^2$, $a = 10$, $w_0 = 1$ 。收敛要求 η 小于 $2/a = 0.2$ 。

学习率决定一切

过小: 方向对, 但 26 步仍没降下来

合适: 几步收敛到接近 0

过大: 两侧来回跳; 再大就直接发散

学习率是最值得优先调的超参数。其他都调对了但它错了, 模型不会收敛。

GD、SGD、Minibatch SGD: 一个关于代价的权衡

	GD 全批量	SGD 单样本	Minibatch SGD
梯度用多少样本	全部 N 个	随机 1 个	一个 minibatch, B 个
单步计算量	$O(N)$, 大数据集不可接受	$O(1)$, 最便宜	$O(B)$, 可控
梯度质量	精确, 方向稳定	方差大, 抖动剧烈	方差 σ^2/B , 可调
硬件利用	可并行但一步太贵	无法利用 GPU 并行	矩阵运算填满 GPU, 效率最高
逃离鞍点	困难, 梯度为零即停	噪声帮助逃离	兼顾, 可用 B 调噪声强度

深度学习里说的「SGD」几乎总是指 Minibatch SGD。B 常取 32 / 64 / 128 / 256。

SGD 的噪声：既是缺点，也是特性

噪声从哪来

minibatch 梯度是无偏估计： $E[\nabla L_B] = \nabla L$

但有方差，正比于 $1/B$ ：批量翻倍，标准差降约 $\sqrt{2}$ 倍

缺点

更新方向抖动，收敛路径曲折

最小值附近无法真正停下，只在噪声决定的邻域内徘徊

—— 这是必须做学习率衰减的根本原因

优点

噪声让参数跳出鞍点平台与狭窄的尖锐极小

尖锐极小盆地窄，容易被踢出；

平坦极小盆地宽，踢不出 → SGD 天然偏好平坦解

理解现代深度学习的一个关键转折：

SGD 只是「因为算不动全量梯度而被迫采用的近似？

它本身就是一种隐式正则化。

把 B 开得过大、噪声过小，反而经常损害泛化。

批量大小怎么定：它和学习率是绑在一起的

批量增大

梯度方差 σ^2/B 变小，方向更可信

可以、也应该同步调大学习率

批量减小

梯度噪声更大，方向不可信

必须调小学习率，否则容易发散

线性缩放规则

批量乘 k ，学习率也乘 k 。

让 ImageNet 训练从数天压缩到 1 小时。

Goyal et al. (2017). arXiv:1706.02677

它为什么会失效

训练最初参数变化极快，直接用放大后的学习率会立刻发散 → 需要 **warmup**（前几个 epoch 线性升温）

批量超过某个阈值后，继续增大不再省时间，泛化开始变差

改进 SGD 的两条独立路线

$\theta_{t+1} = \theta_t - \eta_t g_t$ 里只有两个可动的东西。

路线 A | 改步长 η_t

学习率衰减

Step / Linear / Cosine · Warmup · Cyclical

逐参数自适应

AdaGrad → RMSProp → Adadelta

路线 B | 改方向 g_t

动量

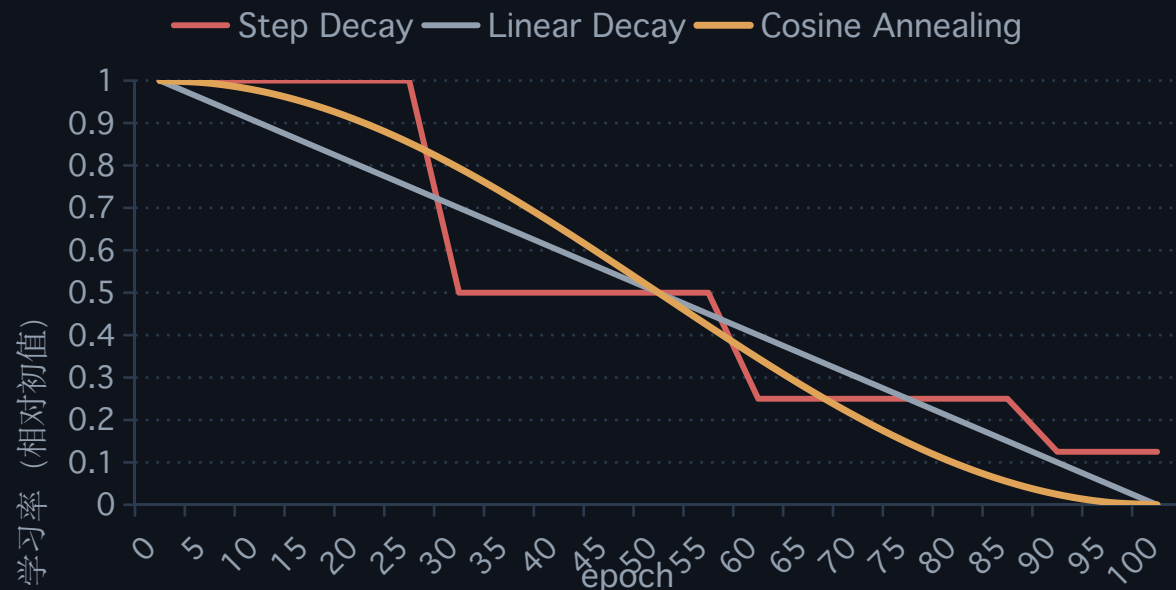
Momentum · Nesterov 加速梯度

限幅

梯度裁剪 (by value / by norm)

两条路线正交，可以同时用。Adam 就是路线 A 的 RMSProp 与路线 B 的动量直接组合

路线 A: 学习率衰减



按各调度公式绘制。Step Decay 取每 30 epoch 减半。

为什么必须衰减

SGD 在最小值附近只能在半径正比于 η 的邻域徘徊

要真正逼近最小值就必须降 η —— 这是收敛性要求，不是调参技巧

怎么选

Cosine: 当前默认，末段平缓，无需人工设台阶

Step: 经典、可控，但台阶位置要试

几乎总配 warmup: 前 5% 步数线性升温

另一条思路: 不降学习率，改为逐步增大批量。

路线 A 进阶：让每个参数有自己的学习率

动机：稀疏特征参数很少收到梯度，应走大步；频繁更新的应走小步。

AdaGrad

累加全部历史梯度平方

适合稀疏特征（推荐系统 embedding），深度网络上少单独用

$$s_t = s_{t-1} + g_t^2$$

$$\theta -= \frac{\eta}{\sqrt{(s_t) + \epsilon}} g_t$$

s_t 单调增 $\rightarrow$ 学习率降到 0，训练过早停止

RMSProp

换成指数加权移动平均，只记最近的梯度尺度

Adam 的一半血统。 ϵ 常取 10^{-8}

$$s_t = \gamma s_{t-1} + (1 - \gamma) g_t^2$$

$$\theta -= \frac{\eta}{\sqrt{(s_t) + \epsilon}} g_t$$

修复过早衰减， γ 常取 0.9

共同点：分母是梯度的二阶矩，起逐参数归一化尺度的作用。

路线 B: 动量, 把历史方向累积起来

更新式

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

β 常取 0.9, 等效于平均最近约 $1/(1-\beta) = 10$ 步的梯度

物理直觉

小球带惯性滚下山坡: 同向分量被累加放大, 来回震荡的分量相互抵消

它解决的三个问题

01 峡谷型曲面上的来回震荡

02 鞍点平台区走得太慢

03 minibatch 噪声导致的方向抖动

Nesterov 加速梯度 NAG

先按动量走到 $\theta_t - \eta\beta v_{t-1}$, 在那个位置求梯度, 再更新

相当于「先看一眼要冲到哪, 再决定怎么修正」——接近极小点时能提前刹车, 减少过冲

Adam: 把两条路线合起来

01 $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$

一阶矩: 动量 (路线 B)

02 $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

二阶矩: RMSProp (路线 A)

03 $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$

偏差修正 (下一页推导)

04 $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{(\hat{v}_t) + \epsilon}}$

一阶矩定方向, 二阶矩定尺度

默认值 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ 在极广的任务上都能用 —— 这正是 Adam 流行的主因。

深入：偏差修正是怎么推出来的

$v_0 = 0$ 导致训练初期 v_t 被严重低估。低估多少可以精确算出。

01
$$v_t = (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} g_i^2$$

展开递推式， $v_0 = 0$ 无额外项

02
$$\mathbb{E}[v_t] = \mathbb{E}[g^2] (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i}$$

设 g_i^2 平稳，期望提到求和号外

03
$$\mathbb{E}[v_t] = \mathbb{E}[g^2] (1 - \beta_2^t)$$

等比数列求和

04
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \Rightarrow \mathbb{E}[\hat{v}_t] = \mathbb{E}[g^2]$$

除掉因子，重新无偏

数量级： $\beta_2 = 0.999$ 时 $t = 1$ 的修正因子是 0.001。不修正则第一步分母小 1000 倍，步长直接爆炸。因子随 t 迅速趋于 1，只在最初几百步起作用。

所有优化器都是同一个模板的填空

$$\theta_{t+1} = \theta_t - a_t d_t$$

优化器	有效步长 a_t	更新方向 d_t	改动了什么
SGD	η	g_t	基线
SGD + Momentum	η	$\beta v_{t-1} + g_t$	改方向
AdaGrad / RMSProp	$\eta / (\sqrt{s_t} + \varepsilon)$	g_t	改步长（逐参数）
Adam	$\eta / (\sqrt{\hat{v}_t} + \varepsilon)$	$\hat{m}_t$	两者都改

实务：先用 AdamW；
CNN 视觉任务调好的 SGD+Momentum 常泛化更好。

AdamW 把 weight decay 从梯度里拆出来单独施加

参数初始化：为什么不能全设成 0

对称性问题

同层权重相同 → 所有神经元输出相同

→ 收到的梯度也相同 → 更新后仍相同

结果：无论多少个神经元，有效宽度退化为 1

必须随机初始化 —— 随机性在这里是为了打破对称，不是引入噪声。偏置可安全置 0。

随机也要选对尺度

太小：信号逐层衰减，深层收不到有效输入，梯度也消失

太大：信号逐层放大，激活进入饱和区，梯度爆炸

需要的是让信号方差逐层保持不变

常见做法

随机初始化：零均值高斯 / 均匀，方差按下一页规则

正交初始化：随机高斯矩阵做 SVD 取正交阵，保证变换不改变长度

预训练初始化：直接用大规模数据上训好的参数 —— 当前最常见也最有效

深入: Xavier 与 He 初始化的方差是怎么算的

前向传播的方差要求

设 $z = \sum_{i=1}^{n_{in}} w_i x_i$, w_i 与 x_i 独立零均值, 则

$$\text{Var}(z) = n_{in} \cdot \text{Var}(w) \cdot \text{Var}(x)$$

要 $\text{Var}(z) = \text{Var}(x)$, 只需 $\text{Var}(w) = 1/n_{in}$

反向传播给出另一个要求

同样推导作用在梯度上, 得 $\text{Var}(w) = 1/n_{out}$

两者一般无法同时满足, Xavier 取折中 $\text{Var}(w) = \frac{2}{n_{in} + n_{out}}$

Glorot & Bengio (2010), AISTATS; He et al. (2015), ICCV.

ReLU 要再改一次

Xavier 默认激活在零点附近近似线性 —— Sigmoid / Tanh 满足, ReLU 不满足

ReLU 把负半轴全置零, 丢掉一半方差: $\text{Var}(z) =$

$$\frac{1}{2} \text{Var}(w) \text{Var}(x)$$

补偿因子 2, 得到 He 初始化:

$$\text{Var}(w) = \frac{2}{n_{in}}$$

用错的后果: 深层 ReLU 用 Xavier, 激活值逐层衰减约 $(1/\sqrt{2})^L$, 30 层后剩约千分之一。

别忘了输入端：数据归一化

初始化管权重尺度，归一化管输入尺度，两者是同一问题的两端。

Min-Max 归一化

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

压到 [0,1]。对异常值敏感。图像除以 255 属此类。

标准化

$$x' = \frac{x - \mu}{\sigma}$$

零均值单位方差。最常用，对异常值更稳健。

白化 / PCA

去相关后再缩放

还去掉特征间相关性。计算贵，深度学习少用。

不归一化：一维 [0,1]、另一维 [0, 10⁴] → 等高线拉成窄峡谷，梯度下降来回震荡

μ 、 σ 只能从训练集统计，再套用到验证/测试集，否则信息泄漏

模块 04 小结

01 我们优化的是经验风险

它是期望风险的无偏估计，但只对固定参数成立

02 高维里的敌人是鞍点

不是局部极小；难点是平台区走得慢

03 平坦极小泛化更好

SGD 噪声天然偏好平坦解，是隐式正则化

04 学习率最重要

必须衰减，且与批量大小绑定（线性缩放）

05 优化器 = 步长 + 方向

Adam = RMSProp（步长）+ 动量（方向）

06 初始化要打破对称并保尺度

ReLU 用 He, Tanh / Sigmoid 用 Xavier

现在损失能降下去了。但训练损失低，不等于泛化好。

练习 4：优化

Q1 为什么偏好平坦的极小值？给出一个不依赖图形的解释。

训练曲面与测试曲面之间存在小偏移。用二阶泰勒展开写出损失变化量，看它与 Hessian 的哪个量有关。

Q3 Adam 的偏差修正如果去掉，训练第一步会发生什么？

代入 $t=1$ ，写出未修正的 m_1 与 v_1 ，算 $m_1 / \sqrt{v_1}$ 的量级。注意 β_1 与 β_2 取值不同，低估程度也不同。

Q2 批量从 64 调到 512，学习率该怎么动？有什么副作用？

先用线性缩放规则给数值。再想两个副作用：初期是否会发散（怎么补救），以及噪声减少对泛化的影响。

Q4 「Adam 收敛更快，所以永远该用 Adam」，这话哪里不对？

区分「训练损失下降快」与「测试性能好」。回到 SGD 噪声与平坦极小。再想 AdamW 为什么需要被单独提出来。

05

训得动之后，怎么训得好

REGULARIZATION

本节要回答

01

训练误差为 0 的模型，为什么有时反而泛化得更好？

02

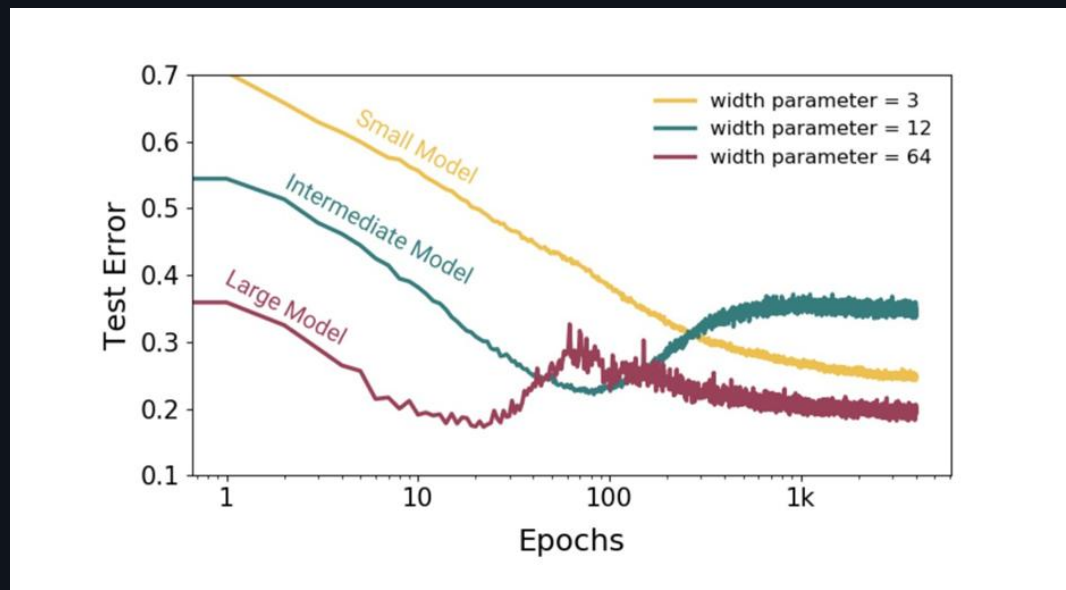
这么多正则化方法，背后是同一件事吗？

03

Batch Normalization 真的是在减少内部协变量偏移吗？

过拟合：现象与三个来源

拟合得过于贴合训练数据，把噪声也学了进去。



不同宽度的 ResNet18，测试误差随训练轮数变化。

三个常见来源

01 训练数据不足

样本没覆盖真实分布，只能记住这几个点

02 参数量远超样本量

过参数化，有足够自由度逐点记忆

03 数据本身有噪声

标签错误、测量误差；模型分不出规律与噪声

注意图中中等宽度后期反而变差，最大的一直在改善
—— 与教科书预期相反。

深入：一个动摇了整套理论的实验

实验设计

CIFAR-10 标签全部随机打乱，图像与标签再无关系

用标准 CNN、标准流程去拟合

结果

训练误差照样降到 0，网络记住了 5 万个随机标签

同一网络、同一超参数，在真实标签上则泛化良好

它推翻了什么

VC 维、Rademacher 复杂度这类只看模型族大小的度量全部失效 —— 两次实验容量完全相同，泛化却天差地别

结论：泛化不可能只由模型结构决定，必然还取决于数据与训练过程

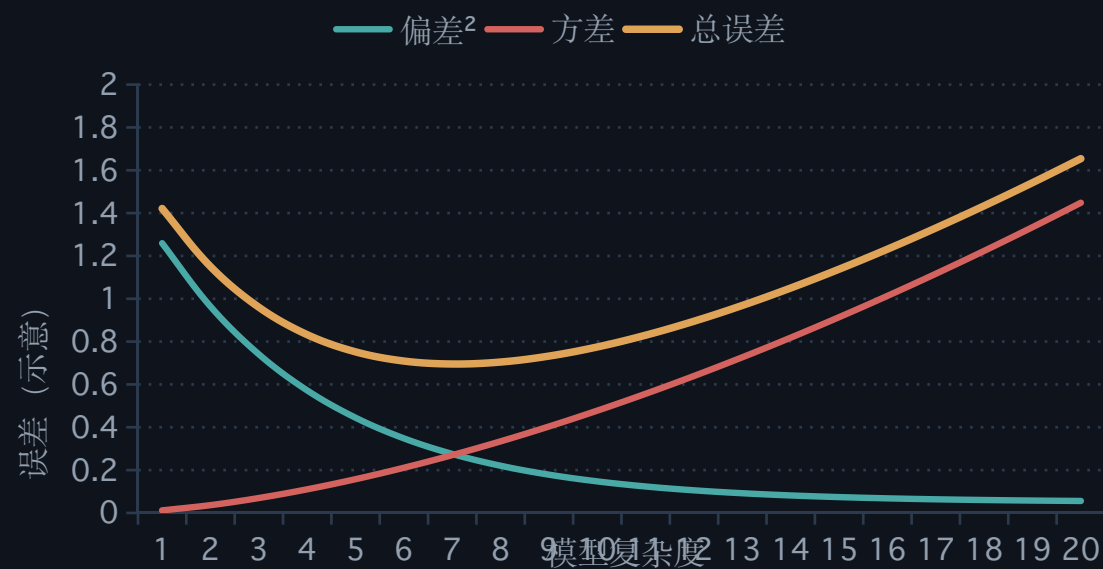
被广泛接受的解释方向：SGD 有隐式偏好 —— 在能拟合数据的无数组参数中，倾向于先找到简单的那些

Zhang, C. et al. (2017). ICLR.

经典框架：偏差-方差分解

$$\mathbb{E}[(t - \hat{y})^2] = (\mathbb{E}[\hat{y}] - f)^2 + \mathbb{E}[(\hat{y} - \mathbb{E}[\hat{y}])^2] + \sigma^2$$

总误差 = 偏差² + 方差 + 不可约噪声



示意曲线，表达趋势，非具体实验数据。

三项各是什么

偏差 Bias

表达能力不足，系统性偏离真实函数 → 欠拟合

方差 Variance

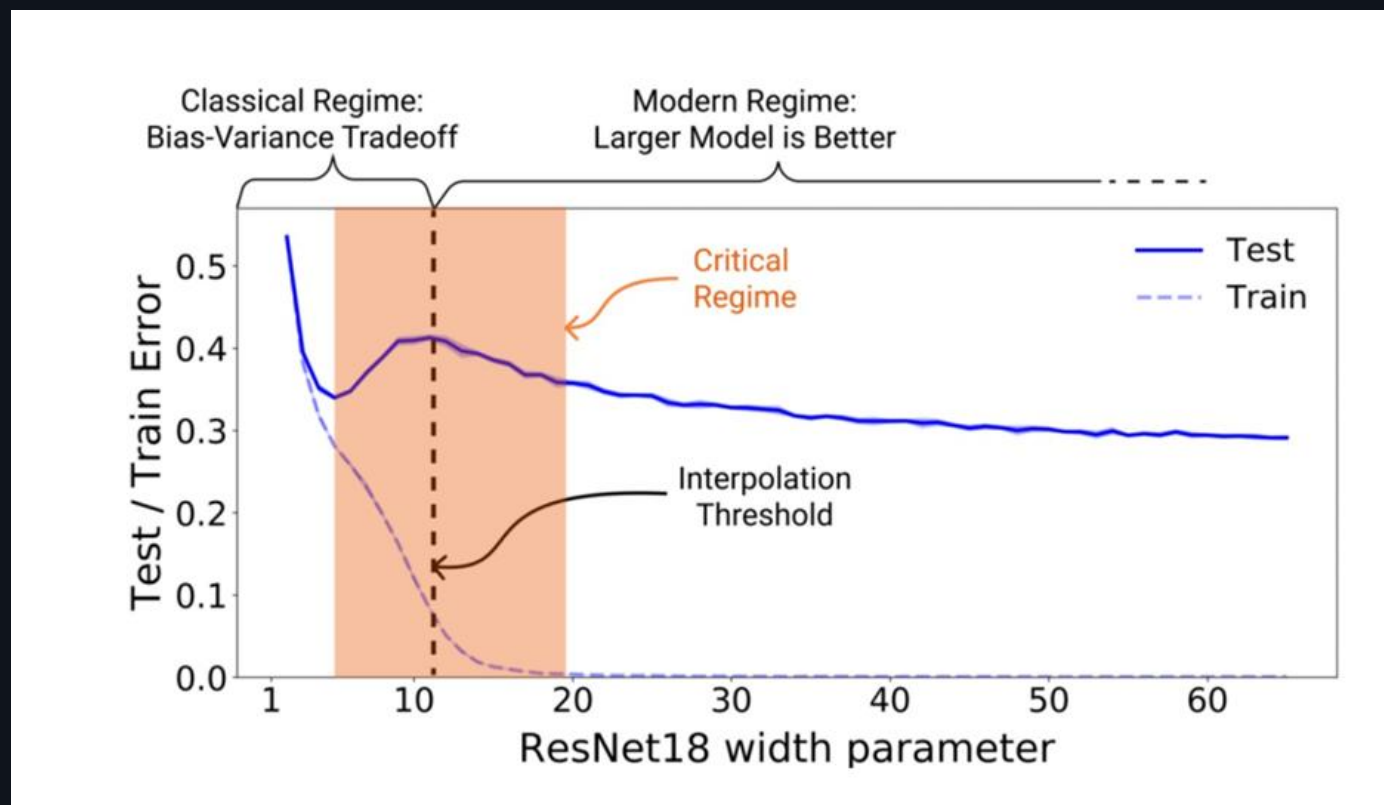
换一批训练数据模型就变样 → 过拟合

噪声

数据生成自带，任何模型都消不掉 → 误差下界

正则化的本质：主动加一点偏差，换方差大幅下降。

深入：经典曲线在深度网络上被打破了



图来自本课程原始讲义，对应 Nakkiran et al. (2020), ICLR。横轴为 ResNet18 宽度。

三个区域

经典区

宽度小时按偏差-方差权衡呈 U 型 —— 教科书那一段

临界区

参数量刚够记住全部训练数据（插值阈值），测试误差局部最高

现代区

继续增大，测试误差重新下降，低于经典区最优点

「模型越大越容易过拟合」只在临界阈值左侧成立。

正则化方法地图：按作用位置分类

方法很多，作用位置只有四个。

改目标函数

L1 / L2 正则

Label Smoothing

改参数更新

Weight Decay

AdamW 解耦衰减

改训练过程

Dropout

Early Stopping

SGD 噪声（隐式）

改数据与表示

数据增强

Batch / Layer Norm

共同本质：缩小模型的有效搜索空间，让能拟合训练集的解里，简单的那部分更容易被找到。

使用原则：不要一次全上。先加数据增强 + weight decay，看验证误差再决定是否叠加。

L1 与 L2 正则：形式相近，效果完全不同

$$\tilde{L}(\theta) = L(\theta) + \lambda \|\theta\|_p^p, p \in \{1, 2\}$$

L2 正则

$$\lambda \sum_j \theta_j^2$$

梯度 $2\lambda\theta_j$ ，正比于自身大小

所有权重按比例缩小，趋近 0 但不等于 0

又叫 weight decay，深度学习默认

L1 正则

$$\lambda \sum_j |\theta_j|$$

梯度 $\lambda \text{sign}(\theta_j)$ ，与大小无关

小权重被直接推到 0，产生稀疏解

常用于特征选择

L1 为什么稀疏：几何解释

等价看成带约束优化：在 $\|\theta\|_p^p \leq C$ 内最小化原损失。

L2 的约束区是圆，处处光滑；L1 是菱形，顶点落在坐标轴上。

等高线由外向内收缩，最先接触处即解。

碰圆 → 落在光滑处（各分量非零）；

碰菱形 → 大概率落在顶点，而顶点意味着除一个分量外全为 0。维度越高顶点越多。

深入：weight decay 与 L2 什么时候不等价

朴素 SGD 下：等价

把 $\frac{\lambda}{2}\|\theta\|^2$ 加进损失，梯度多出 $\lambda\theta$ ：

$$\theta \leftarrow (1 - \eta\lambda)\theta - \eta g$$

即「每步乘一个略小于 1 的系数」= weight decay，同一件事

这就是现在训练 Transformer 一律用 AdamW 而不是 Adam+L2 的原因。一个看似微小的实现差别，在大模型上差距非常明显。

Loshchilov & Hutter (2019). ICLR.

Adam 下：不等价

Adam 把整个梯度（含 $\lambda\theta$ ）一起除以 $\sqrt{\hat{v}_t}$

后果：历史梯度大的参数 $\sqrt{\hat{v}_t}$ 大，实际衰减反而更弱——而它们恰恰最需要被约束

L2 项在 Adam 下被扭曲，正则效果显著变差

AdamW：不把 $\lambda\theta$ 放进梯度，而在自适应缩放之后单独施加衰减。

Early Stopping: 最省事的正则化

做法

每 epoch 在验证集评估，记录最好的一次；连续若干轮不再下降就停止，回退到最好的参数

唯一超参数：耐心值 `patience`

为什么它算正则化

训练步数本身就是容量：步数越多，参数离初始点越远，可表达的函数越复杂

提前停止 = 限制参数移动的距离。二次损失下可证与 L2 有直接对应

优点

零成本：不改模型、不改损失、不增计算，还省下浪费的训练时间

三个陷阱

必须用验证集，不能用测试集

验证曲线有噪声，耐心值太小会过早停止

与双下降冲突：临界区附近验证误差可能先升后降，早停会正好停在最差位置

实务：几乎总是开着，配合 checkpoint 保存最优权重。

Dropout: 训练时随机关掉一部分神经元

训练时

$$y = f(W(d \odot x) + b), d_i \sim \text{Bernoulli}(1 - p)$$

每次前向独立采样掩码 d ，以概率 p 置零。 p 常取 0.5（全连接）或 0.1（Transformer）

被丢弃的神经元这一步也不接收梯度

推理时

不再丢弃，全部参与。但期望会偏大 $1/(1-p)$ 倍

实现上用 inverted dropout: 训练时就除以 $(1-p)$ ，推理时什么都不做

视角一：集成学习

每次采样掩码 = 从原网络抽出一个子网络训练

n 个神经元共 2^n 个子网络，共享权重

推理时用完整网络 $\approx$ 对这指数多个子网络求平均 —— 集成能降方差

视角二：抑制协同适应

神经元无法依赖特定伙伴（随时可能被丢），只能学独立可用的特征

削弱特征间的脆弱共谋，表示更冗余、更鲁棒

注意：Dropout 与 BN 混用时常互相干扰，顺序需谨慎。

从数据侧下手：数据增强与标签平滑

数据增强 Data Augmentation

对图像施加不改变标签的变换，人为扩充数据量

几何 旋转、翻转、缩放、平移、随机裁剪

色彩 亮度、对比度、饱和度抖动

噪声遮挡 高斯噪声、Cutout、随机擦除

混合 Mixup、CutMix（同时混合图像与标签）

本质：把「模型应当具备的不变性」直接喂给模型。翻转不变性是先验知识，增强只是把它写进数据。

标签平滑 Label Smoothing

$$t'_k = (1 - \varepsilon) t_k + \frac{\varepsilon}{K}$$

one-hot 的 1 换成 $1 - \varepsilon$ （如 0.9），其余分摊 ε/K 。 ε 常取 0.1

动机：one-hot 要求输出「100% 确定」，这需要 logit 差趋于无穷大——模型会不断放大权重去逼近一个永远达不到的目标

效果：抑制过度自信，改善置信度校准（预测 90% 时实际正确率也近 90%），同类表示更紧凑。代价：需要精确概率或做知识蒸馏时可能有害。

Batch Normalization: 把每层输入重新标准化

模块 04 对输入数据做标准化；BN 的想法是对每一层的输入都做一次。

01 $\mu_B = \frac{1}{B} \sum_{i=1}^B z_i$ minibatch 内、逐特征求均值

02 $\sigma_B^2 = \frac{1}{B} \sum_{i=1}^B (z_i - \mu_B)^2$ 同样逐特征求方差

03 $\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{(\sigma_B^2 + \epsilon)}}$ 标准化为零均值单位方差

04 $y_i = \gamma \hat{z}_i + \beta$ 再用可学习的 γ, β 缩放平移

第 4 步关键：强制每层输出都是标准正态会限制表达能力， γ, β 让网络可以学回任意均值方差，甚至学成恒等映射。

训练与推理不一致

训练用当前 batch 统计量；推理没有 batch，改用训练期累积的滑动平均

所以 BN 有训练/评估两种模式 —— 忘记 `model.eval()` 是最常见的线上事故之一

附带的正则效果：同一样本在不同 batch 里被标准化的结果不同，相当于给激活注入噪声。

Layer Normalization: 换一个方向做归一化

公式完全相同，唯一区别是「在哪个维度上求均值和方差」。

	Batch Normalization	Layer Normalization
归一化方向	跨样本，逐特征 (batch 维)	跨特征，逐样本 (feature 维)
是否依赖 batch	是，batch 太小统计量不可靠	否，单样本即可
训练/推理一致性	不一致，需滑动平均	完全一致，无额外状态
变长序列	处理困难	天然适配
典型应用	CNN、图像任务	Transformer、RNN、NLP

还有 Instance / Group / Weight Norm，区别同样只在于「沿哪些维度统计」。

深入：BN 有效的原因，和最初的解释不一样

原论文的解释：减少内部协变量偏移

前层参数一变，后层收到的输入分布就变（ICS），迫使后层不断重新适应

BN 把每层输入分布固定住，于是训练更快更稳

这个解释直观、流传极广，写进了无数教材 —— 但后来的实验并不支持它。

一个方法有效，不等于人们对它有效原因的第一个解释就是对的。

反驳性实验

在 BN 层后人为注入随时间变化的噪声，故意制造严重分布偏移

结果：ICS 明显变大，训练速度几乎不受影响，仍远快于无 BN

说明减少 ICS 不是 BN 起作用的原因

更站得住的解释

BN 让损失及其梯度的 Lipschitz 常数变小 —— 损失曲面被显著平滑

曲面更平滑 → 梯度方向在更大范围内可信 → 可用更大学习率

Santurkar et al. (2018). NeurIPS.

模块 05 小结

01 经典图景不完整

随机标签与双下降说明：容量本身不能决定泛化

02 偏差-方差仍有用

正则化 = 加偏差换方差；但只覆盖临界阈值左侧

03 所有正则化都在缩小搜索空间

改目标、改更新、改过程、改数据

04 L1 稀疏、L2 收缩

几何上是菱形顶点 vs 圆的光滑边界

05 Adam 下 $L2 \neq \text{weight decay}$

用 AdamW，把衰减从自适应缩放中解耦

06 BN 的真实机理是平滑曲面

不是减少内部协变量偏移

五个模块到此结束。最后用十分钟把整条主线串一遍。

练习 5：泛化与正则化

Q1 L1 为什么产生稀疏解，L2 为什么不会？用几何图形说明。

画出二维参数平面上的约束区域（菱形 vs 圆）与损失等高线，标出相切点。再从梯度角度验证：两者在 $\theta \rightarrow 0$ 附近有什么本质差别？

Q3 训练损失 0.01、测试损失 2.5。列出你会依次尝试的三件事，说明顺序理由。

典型高方差。候选：更多数据 / 数据增强、加正则、减小模型、早停。为什么先做数据侧而不是先减小模型？

Q2 用 Adam 训练时加了 L2 正则却几乎没效果，问题出在哪？

写出 Adam 更新式，看 $\lambda\theta$ 经过了什么。哪些参数受到的实际衰减被削弱了？改用什么优化器可以修复？

Q4 既然大模型在现代区反而泛化更好，是不是就不需要正则化了？

回到双下降图，注意横轴是模型宽度不是数据量。若数据量固定而模型无限增大，测试误差会降到 0 吗？

Q1、Q2 需要动笔。Q3 是面试高频题，建议按「先诊断、再干预」组织回答。

把整条主线串一遍

如果今天只记住一件事，记住这五步之间的因果关系，而不是任何一个单独的公式。

01	泛化	我们只能看见训练误差，却要控制测试误差	容量、数据量、约束强度三者平衡
02	交叉熵	把分类写成最大似然，损失函数就被推导出来了	梯度 $\hat{y} - t$ ，无饱和
03	深度网络	线性模型表达力不够，让基函数变得可学	代价是非凸、梯度连乘
04	优化	非凸曲面上只能迭代下降	学习率 + 方向；鞍点与平坦极小
05	正则化	训练损失低不等于泛化好	缩小有效搜索空间

| 每一步的存在，都是因为上一步留下了一个没解决的问题。

今天的内容接到哪里去

本课讲的是所有深度模型共用的地基。后续课程都是在这个地基上换结构。

CNN

卷积神经网络

把「平移不变性」这个先验直接写进结构，而不是靠数据增强去教。参数共享大幅降低了参数量。

Transformer

注意力机制

用注意力替代固定的连接结构，让每个位置自行决定关注谁。LayerNorm、AdamW、warmup 全部来自今天的内容。

生成模型

VAE、扩散模型

从「预测标签」转向「建模数据分布」。最大似然的思路与模块 02 完全一致，只是似的对象换了。

一个建议：遇到新模型时，先问它在这五个模块的哪一层做了改动。绝大多数「新方法」只动了其中一层。

课后阅读：按模块对应到章节

主教材：Bishop, C. M. & Bishop, H. (2024). Deep Learning: Foundations and Concepts. Springer.

本课模块	Bishop (2024)	Goodfellow et al. (2016)	重点概念
模块 01 · 泛化	Ch. 1.1–1.2, Ch. 4	Ch. 5 (机器学习基础)	多项式拟合、模型选择、偏差方差
模块 02 · 交叉熵	Ch. 5.4, Ch. 2–3	Ch. 6.2 (输出单元与损失)	最大似然、信息论、指数族
模块 03 · 深度网络	Ch. 6	Ch. 6 (前馈网络)	通用近似、激活函数、反向传播
模块 04 · 优化	Ch. 7	Ch. 8 (深度模型的优化)	SGD、动量、自适应方法、初始化
模块 05 · 正则化	Ch. 9	Ch. 7 (正则化)	L1/L2、Dropout、归一化

补充：《Dive into Deep Learning》（动手学深度学习）提供了本课全部内容的可运行代码，建议配合第 3–5、11 章阅读。

THANK YOU

问题时间

值得你自己去想的三个问题:

为什么过参数化的模型反而泛化更好?

如果测试数据来自另一个分布, 今天的结论还剩多少?

课件将在课后发布。

魏鸿鑫

南方科技大学

统计与数据科学系